

FYP25CM009
Simulation of a track car

Glen Kelly

22443026

Final Year Project – 2026

B.Sc. Single Honours in
Computer Science



**Maynooth
University**

National University
of Ireland Maynooth

Department of Computer Science
Maynooth University
Maynooth, Co. Kildare
Ireland

A thesis submitted in partial fulfilment of the requirements for the
B.Sc. Single Honours in Computer Science.

Supervisor: Dr Charles Markham

Contents

Declaration	ii
Acknowledgements	iii
Abstract	iv
Chapter 1: Introduction of the Project	5
1.1 Overview of Track Car Simulation	5
1.2 Motivation	5
1.3 Approach	5
1.4 Overview of the rest of the report.....	6
Chapter 2: Background Research	7
2.1 Simulation in Motorsport.....	7
2.2 Review of Existing Simulations	7
2.3 Overview of the physics of car dynamics.....	8
2.4 Formulae and Concepts	9
2.5 Technology.....	9
2.5.1 Java	9
2.5.2 Processing.....	10
2.5.3 Spline Interpolator	10
2.5.4 Polynomial Spline Function	10
2.5.5 Univariate Function	10
2.5.6 Java.io.*	10
2.5.7 org.apache.poi.*	10
2.6 Statistical Formulae	11
2.6.1 Mean Average.....	11
2.6.2 Standard Deviation.....	11
2.6.3 Coefficient of variance	11
2.7 Reflection before Development	11
Chapter 3: Implementation of Simulation	12
3.1 Simulation to be created	12

3.2	User Case	12
3.3	Analytical Work	15
3.4	Architectural Level.....	15
3.5	Software Interactions	17
3.5.1	User Interface/Experience (UI/UX).....	17
3.5.2	Excel.....	17
3.6	Track Description.....	20
3.7	Spline	20
3.8	Lap Times and Physics	20
3.9	Addition of Genetic Algorithm to find race line.....	21
3.9.1	Description.....	21
3.9.2	AI and Prompts	21
3.9.3	Building on Framework.....	21
3.9.4	Integration.....	22
3.9.4	Final Product.....	22
3.10	Race Stints and Tyre Degradation.....	22
3.11	Statistical Data	22
Chapter 4: Evaluation.....		23
4.1	Metrics of Success	23
4.2	Comparison with Circle Solution.....	23
4.3	Comparison with Real Tracks	25
4.4	Comparison with Real Lap Times.....	26
4.5	Software Verification and Testing	26
4.5.1	Testing and Verification Approach	26
4.5.2	Circle Verification	26
4.5.3	Checking Points Test.....	26
4.5.4	Checking Radius Calculations Test	26
4.5.5	Checking Speed Calculations Test	26
4.5.6	Checking Lap Time Progression Test.....	27
4.5.7	Checking Mean and Standard Deviation of Genetic Algorithm	27

4.5.8	Checking against existing data	27
4.5.9	Interpretation of the results	27
Chapter 5: Conclusion.....		28
5.1	Achievements.....	28
5.2	Project Reflection	28
5.3	Future Work.....	29
5.3.1	Framework and Improved Physics.....	29
5.3.2	Stand-Alone Strategy video game	29
5.3.3	Integration with a simulation game	29
Bibliography		30
GitHub Repository		33
Appendices		34
Appendix 1: Imagery, Diagrams and Figures		34
Appendix 2: Equations Used in the Project.....		42
2.1,2.2,2.3	Kinematic Equations	42
2.4	Acceleration Definition.....	42
2.5	Area under the velocity–time graph	42
2.6	Determinant.....	42
2.7,2.8	Circle Centre	42
2.9	Distance between two points.....	43
2.10	Max Speed around a curve.....	43
2.11	Max Speed	43
2.12	Arc Length Accumulation	44
2.13	Distance, Speed and Velocity Formula	44
2.14,2.15	Parametric Spline	44
Appendix 3: Use of AI		45
A2.1	Summary of AI usage	45
A2.2	Prompts	46

A3.1 Code Snippets.....	51
-------------------------	----

Declaration

I hereby certify that this material, which I now submit for assessment on the program of study as part of the B.Sc. The Single Honours in Computer Science qualification is *entirely* my own work and has not been taken from the work of others, save to the extent that such work has been cited and acknowledged within the text of my work.

I hereby acknowledge and accept that this thesis may be distributed to future final-year students as an example of the standard expected of final-year projects.

Signed



____ Date: 15/03/2026

Acknowledgements

I would like to take a moment to acknowledge and thank several people for their support and motivation throughout the project.

- Dr Charles Markham for his enthusiasm, support, and advice throughout the project. His background and expertise in Physics and Computer Science were an outstanding guidance throughout the project. Alongside this, he was fundamental in helping assess the project's progress and ambition, helping me to push the project to its maximum potential.
- My peers and friends with whom I spent my time throughout my fourth year, acting as sounding boards for ideas, offering proofreading and great company for a coffee break.
- My family, friends and wonderful girlfriend who believed in me throughout my second attempt at a bachelor's degree, the project as a whole and helped me keep on the right path.

Abstract

This report outlines the development of a race car strategy tool. The tool is designed to take as an input a description of both the track and the car. The tool then provides an estimate of the lap times expected on the real track. The tool can identify the optimum racing line on the track and display where the car traverses on its way to that lap time. The software developed takes advantage of Genetic Algorithms to generate the best possible racing line given the conditions set in the simulation.

The approach taken during the development led to the creation of a software framework which can be extended to support extra functions in the future. This modularised approach also allows for efficient flexibility of the software and its data.

In addition, an intuitive and straightforward interface was developed for the simulator to allow for instant alterations to parameters within the simulation. This allows for more efficient data gathering potential for the user.

This code was developed in Java and took advantage of Processing for its graphics engine to display the UI/UX. This provided a stable platform for rapid prototyping and assessment.

Several commissioning experiments are presented to demonstrate the operation of the simulator. This includes track data from Brands Hatch Indy Circuit, Bahrain International GP circuit and the BMW M4 GT4.

Chapter 1: Introduction of the Project

This chapter outlines the topic of race car simulation. It provides the motivation behind the software developed and how development is approached to identify the optimum racing line

1.1 Overview of Track Car Simulation

“There are quite a few things that are not easy to model... A lot of what you do is estimation.” (Collins, 2024, p132). As this quote suggests, simulation is not an exact science; however, when done correctly, it can provide valuable insights without the complexity and challenges of testing a car on a track. (Balci, 1995). The objective of this project is to digitally recreate a simulation of a track car that includes a physics model as the backbone of the calculations. (O'Regan, 2000).

This project aims to replicate real-life racing circuits by breaking them down into a set number of gates and choosing where in the gate to traverse to find the optimal line of a car around the circuit. This is similar to the software found in Formula 1™ (F1) for driver coaching. (McLaren, 2013). The project should produce a set of times around the track that will improve upon each simulation, with diminishing improvements as the software runs for longer. The lap time will be defined by the virtual drivers' steering, braking and acceleration inputs, replicating real life as closely as possible.

1.2 Motivation

The motivation of this project is to show the characteristics of the “Perfect Lap.” The software will show how car control (acceleration and braking), the characteristics of a turn, and track grip, can all create the perfect fastest lap.

In Bernie Collins' book “How to Win a Grand Prix” (Collins, 2025) Bernie refers to the software used in high-level motorsport and how it can run thousands of iterations for any given race to find the optimal qualifying lap. The software will be built modularly to allow future expansion. This might include full Grand Prix simulations, the effects of tyre wear, and adding a large pool of cars to the simulation.

1.3 Approach

A software engineering approach was taken to the development of the code. This involved initially creating user stories and personas during the research phase. The user stories and personas were then used to create a wireframe description of the simulator.

Following the system design, the software was developed. This included developing data structures to describe the track. This was using a set of waypoints, between which a cubic spline was interpolated. The cubic spline provides a nearly continuous representation of the track's centreline. To give the track width, entry and exit gates were used at a set width. The position of the car as it comes through these gates would then determine the path it takes around the track. To aid visualisation of the paths taken around the track, the gate positions are then connected via a spline. This provides a visual of how the car moves relative to the centreline and extremities of the track. The simulation then quantifies the radius of each gate point, which it uses to work out the car's maximum possible velocity at that point. From this data, the simulation can build a lap using the max velocity and the car's acceleration and braking ability. This can then be quantified into a lap time using the distances between the gates and the car's relative speed.

Finally, the simulation can be attached to a genetic algorithm, which will experiment with gate positions to find the best time around the track it can. These laps can then be called in succession to quantify a race stint, in which mass can be adjusted for fuel burn, and tyres can be degraded.

1.4 Overview of the rest of the report

The software developed in this project provides a practical demonstration of how precise driving can help enhance lap time through minuscule adjustments. The simulation should also help the user create an understanding of how elite drivers can unlock lap time with the help of software. The information given can help them aim for points with millimetres as references and nanosecond differences in throttle and brake application.

Chapter 2: Background Research

In this chapter, the background research that was required to create the simulator developed is discussed. This includes the source of inspiration, a brief history and existing software solutions that exist within the motorsport space.

2.1 Simulation in Motorsport

Since the 1980's, in motorsport (particularly Formula 1), the desire to introduce electronic telemetry gathering equipment to race cars has grown. At the beginning, the team's focus was to have an accurate fuel level reader to guarantee their fuel levels throughout a Grand Prix. ("A brief history of computing in Formula 1," 2016).

As technology matured, engineers began introducing thermometers, vibration sensors, and tyre pressure monitors in any area of the car they could. This coincided with the introduction of a live data connection between the car and the pit wall (garage). Drivers now found they were not as alone in driving the car as they once were (McLaren, 2013).

At the turn of the millennium, teams began using simulation tools to digitally run the car across millions of laps. This allowed the teams to see the ideal qualifying lap, the "perfect" race strategy for set conditions and compare this data to what their driver had just done on track. This was the dawn of the era of the Driver Performance Coach. The management of data has become central to the race engineer's duties over a Grand Prix weekend. (Collins, 2025).

This project aims to develop a race-track simulation which looks to current platforms for inspiration. The goal is to keep the model straightforward and effective. Code was developed from the ground up to allow a full exploration of the components of a track simulator.

2.2 Review of Existing Simulations

Many different tools within motorsport are available; however, they are shrouded in secrecy due to the limitations of patents. As Lakhani said, "[B]y the time a patent reaches grant, the technology to which it relates may be virtually obsolete" (Lakhani, 2017).

Catapult developed a software known as 'RaceWatch: Race Analysis Software', which is used in the F1 paddock.¹

("Race Analysis Software | Strategy & Circuit," 2026). According to the website, the tool features "visualisation" and "predicted analysis," and the illustrations used suggest live telemetry from the track as well as an AI-driven strategy analysis element. Bernie Collins wrote a piece for Catapult in 2023, providing the best insight into the tool. "With RaceWatch, teams access detailed telemetry, which provides insights into all aspects of car performance: engine efficiency, transmission, aerodynamics, and more" (Collins, 2023), see figure 2,1.



Figure 2.1 Catapult RaceWatch software in use by BWT Alpine Formula One Team at the 2025 Miami Grand Prix (“RaceWatch in F1,” 2025)

To further interpret the real world, we can turn to sim racing. The phenomenon has come a long way from sticking a wheel to a desk with suction cups. The reigning F1 World Champion, Lando Norris has been quoted as saying “I had a sim at home when I was younger and it provided invaluable help in developing my race craft” (Tudor, 2019). Therefore, justification in creating a comparison between Sim racing and real-life motorsport brings us to Sim Racing Telemetry.

Sim Racing Telemetry (SRT) is used by sim racers as a way of pulling telemetry from games that do not have built-in analysis. They aim to make a realistic telemetry software that allows sim racers to access the same information as real-life teams. “SRT lets every sim racer quickly acquire, analyze and review detailed telemetry data from sim racing games.” (“SRT | Sim Racing Telemetry,” 2026). They promote a game session view with per-lap information, which is like the information available to drivers in real life via the radio or their dashboard. Similarly, after the session, SRT offer lap comparison, statistics, lap charts, and the ability to export to Excel. In their own documentation, they say “the path taken by a driver through a corner or series of corners, is an essential aspect of telemetry data analysis that can lead to significant performance gains” (“Analyzing Racing Lines in Telemetry Data [Sim Racing Telemetry Docs],” 2026). In this document, they also discuss the four main elements of a racing line: “Entry, Apex and Exit,” “Braking Zones”, “ Throttle Application”, and “Steering Input.” These are quantifiable and can be used as metrics for the success of any automated application, such as this project.

2.3 Overview of the physics of car dynamics

Car vehicle dynamics can be quantified as trying to get a mass to move in a desired manner while typically only maintaining four small contact patches with the ground via rubber tyres. Every movement, steering input and change in momentum must be processed via these four contact patches. The grip coefficient of these contact patches becomes the limiting factor of most vehicles. If you wish to corner harshly, the limit at which you will continue in controlled motion is the point at which the forces of the vehicle in motion overcome the coefficient of grip (C.O.G.) and the vehicle begins to slide (See Figure 2.2). Similarly, if you try to accelerate harshly, at the point at which the torque (See Figure 2.3) overcomes the grip co-efficient you stop making efficient forward progress and the wheels spin at a greater rate than motion (See

Figure 2.4). Furthermore, when braking, the limit to the braking power used is also the coefficient of braking. When applying brake pressure inducing more torque than the coefficient of friction of the tyre, the tyre can start to slide, stop spinning and bind with the brake pad (See Figure 2.5). In this scenario, the solution is to release brake pressure until the wheel begins to rotate again.



Figure 2.2 Car sliding. (Jamieson, 2020)

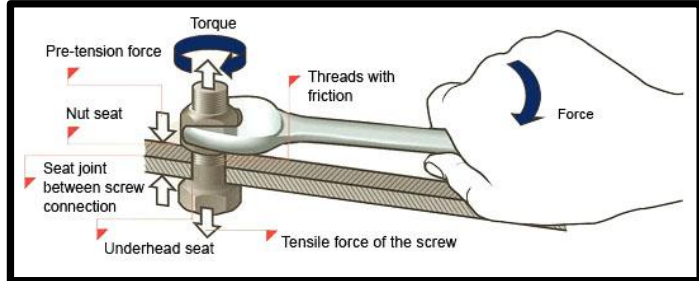


Figure 2.3 Torque Explanation ("What is Torque? | Mountz," 2018)



Figure 2.4 Overcoming C.o.G. and spinning tyre. (Lee, 2020)



Figure 2.5 Brakes overpower C.O.G. (Gaonkar, 2024)

Therefore, if you are trying to make a car go quicker around a circuit, there are only so many compromises you can make, as the limiting factor will always be the grip of the tyre. For example, downforce, in effect, increases the force on top of gravity to increase the coefficient of grip at the contact patch artificially, and aids such as traction control and the anti-lock braking system (ABS) aim to get the most performance from the tyre.

2.4 Formulae and Concepts

The formulae and concepts for this project establish the physics model that runs the simulator. They are fully explained and detailed in the appendix at Appendix 2 and will be referenced throughout chapter 3. These formulae include the kinematic equations, acceleration equation, linear algebra formulae as well as the details of the spline calculations.

2.5 Technology

The Technology section provides an overview of the languages and libraries used in the development of the simulation. Detail is provided as to the reasoning for the choices and the capabilities of the technology in creating the simulation.

2.5.1 Java

Java is chosen for this project as a very capable object-oriented programming language; it provides a platform which is well known. The language has been credited with being "immensely conducive to quality and productivity, and a joy to work with" by one of the creators of Java, and it fits perfectly for a high-stress, time-constrained project (Bloch, 2008). All of this will allow for a higher ceiling of potential, but also, due

to its adaptability and clarity, the code could easily be converted to another language, such as Python.

2.5.2 Processing

Processing provides such a fundamentally solid base for physics-based programming, building on the Java base and allowing for a simple and effective UI to be created. The functions within Processing are well documented on their reference pages and are instinctively straightforward to follow. (“Processing Reference Page,” 2025). The experience of using Processing in Maynooth, as well as its integration with Java, makes it the perfect platform for rapid prototyping for this project’s needs.

2.5.3 Spline Interpolator

`org.apache.commons.math3.analysis.interpolation.SplineInterpolator.`

This Library creates the smooth curve that passes through the points that are created by the user by constructing connected cubic polynomials.

2.5.4 Polynomial Spline Function

`org.apache.commons.math3.analysis.polynomials.PolynomialSplineFunction`

Represents the actual piecewise polynomial curve produced by the interpolator, allowing evaluations at any position on the curve.

2.5.5 Univariate Function

`org.apache.commons.math3.analysis.UnivariateFunction`

Creates a function that defines the relationship between x,y and the distances (s) of points on the curve.

2.5.6 Java.io.*

`java.io.FileOutputStream;`

`java.io.File;`

The java.io library allows for the export of the data from the project to other sources, and in this case, allows for the transfer of the data into the Excel file that contains the complex race simulation data

2.5.7 org.apache.poi.*

`import org.apache.poi.ss.util.CellRangeAddress;*`

`import org.apache.poi.xddf.usermodel.*;`

`import org.apache.poi.xddf.usermodel.chart.*;`

`import org.apache.poi.xssf.usermodel.*;`

`import org.apache.poi.ss.usermodel.HorizontalAlignment;`

The Apache suite of libraries has been utilised in this project to allow the Excel sheet to be built within the code. It provides the structure that is used to define each sheet, what data will be represented and what visualisation will be included.

2.6 Statistical Formulae

In this project, a genetic algorithm is used to calculate the shortest and quickest laps it can for evaluation. To assess both the variability and accuracy of the laps created, the results can be statistically assessed using the following methods.

2.6.1 Mean Average

The mean average is obtained by getting the sum of all data and dividing the result by the number of data points. See equation 2.16.

$$\bar{x} = \frac{\sum_{i=1}^N (x_i)}{N} \quad (2.16)$$

where $\sum$ is the summation symbol, x_i represents each data point, N is the number of data points, and $\bar{x}$ is the mean. (LLC, 2026a)

2.6.2 Standard Deviation

The standard deviation measures data dispersion in relation to the mean. It is calculated by getting the difference between each data point and the mean, then squaring it; these differences are then all summed together. This figure is then divided by the number of data points, and the entire result is square-rooted. See equation 2.17.

$$\sigma = \sqrt{\frac{\sum_{i=1}^N (x_i - \bar{x})^2}{N}} \quad (2.17)$$

where $\sum$ is the summation symbol, x_i represents each data point, N is the number of data points, and $\bar{x}$ is the mean. (LLC, 2026b)

2.6.3 Coefficient of variance

The coefficient of variance is used to measure relative variability and shows how large the standard deviation is compared to the mean. See equation 2.18

$$\text{Coefficient of Variance (\%)} = \left(\frac{\sigma}{\bar{x}}\right) \quad (2.18)$$

σ is the standard deviation, $\bar{x}$ is the mean, and the Coefficient of Variance is expressed as a percentage. (LLC, 2026c)

2.7 Reflection before Development

In this chapter, the theoretical platform and technology for the simulation were explored to take the research forward into development. The data found will provide the backbone of the methods created to run the simulation and will be referred to throughout the Implementation of Simulation chapter.

Chapter 3: Implementation of Simulation

This chapter explores in detail how the software was created. The chapter details how a modular approach was used for code development, and how artificial intelligence was utilised to help with the development of the genetic algorithm creation.

3.1 Simulation to be created

This project aims to create a software solution that simulates a track car navigating a track. This software should be bound by modifiable physics that will dictate the car's ability to accelerate, brake and corner. The car should have free rein to choose its own racing line around the track within the bounds of the racetrack. For clarity, a valid lap time will comply with the F1 sporting regulations. Specifically, article 33.3, which states "Drivers will be judged to have left the track if no part of the car remains in contact with it and, for the avoidance of doubt, any white lines defining the track edges are considered to be part of the track" (Fédération Internationale de l'Automobile, 2024). We will treat the inside and outside lines as the "white lines" defining the edge of the racetrack. In this work, the software adjusts the car speed such that it can never leave the track, meaning that only legal, real-life lap times are comparable.

3.2 User Case

A typical use case for this software revolves around F1 and has been constructed using information gathered in "Total Competition", which breaks down Ross Brown's career in motorsport. The book specifically details the dynamics and roles from his time in Benetton F1, Scuderia Ferrari, Brawn GP and Mercedes AMG Petronas F1 Team. (Ross Brawn and Adam Parr, 2017).

The following user case references a typical case of using the software in a Formula One-style scenario, and **Figure 3.1** is a supporting diagram for this case.

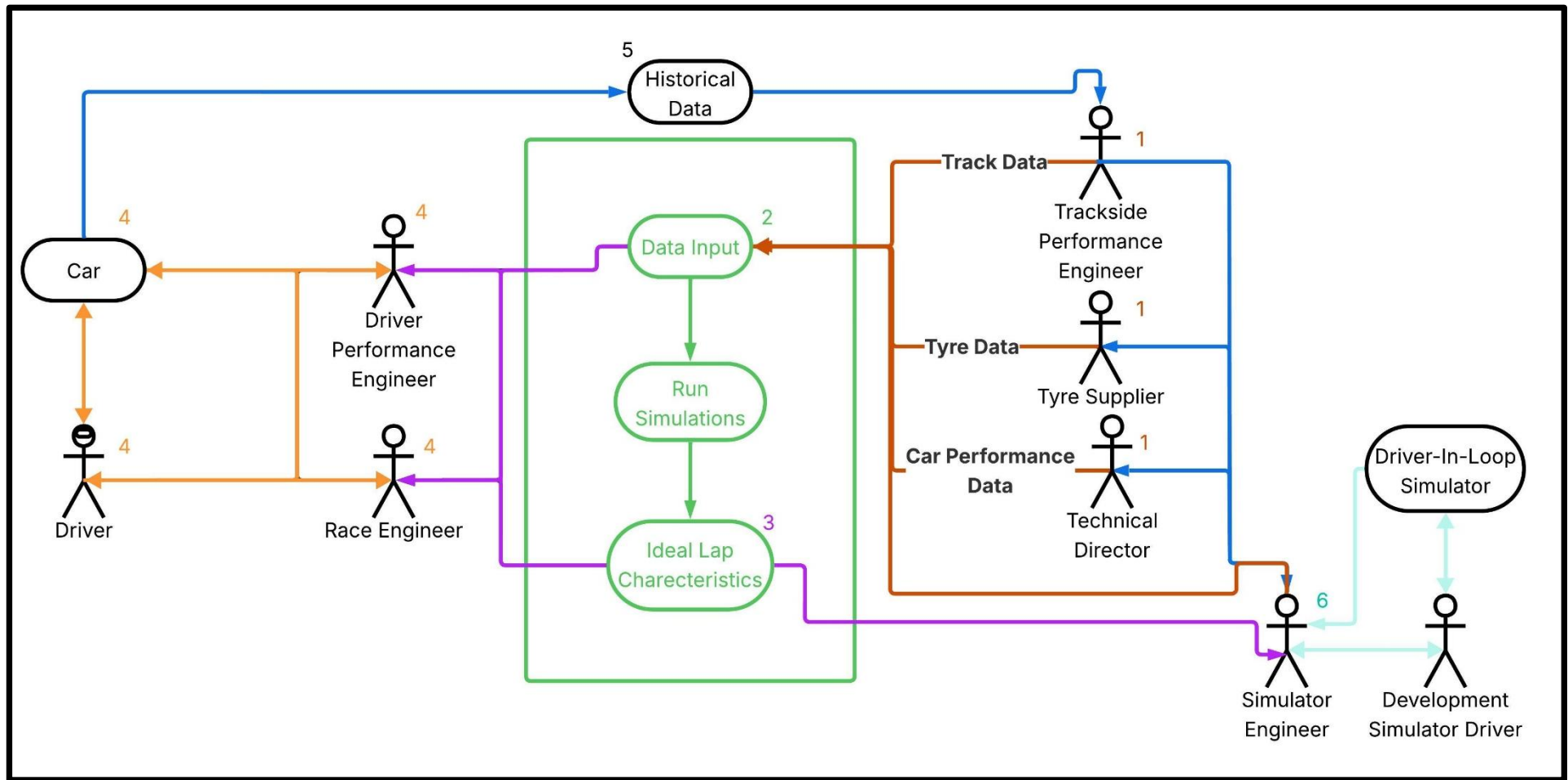


Figure 3.1 Typical Use Case of Existing Software

1. Initially, track data is gathered from the Trackside Performance Engineer. This data includes temperature, windspeed and historical run data from the last visit to the circuit, if available. Tyre and track grip data are provided from the tyre supplier, in F1's case, Pirelli. The data will include a grip trend graph for each type of tyre being brought for the race weekend, as well as the grip component of the track surface at key areas around the track, such as braking zones and medium to high-speed corners. The technical specifications of the car, such as braking power, acceleration and steering ability, are provided by the technical director and the technical team.
2. The data from the team is then fed into the simulation software, which will set up to create telemetry for the best lap time possible in the conditions provided. This data will be conglomerated into a visualised package ready to be analysed.
3. The telemetry package can then be distributed to the team; the key people receiving the data are the Driver Performance Engineer, Race Engineer, and Simulator Engineer.
4. The Race Engineer and Driver Performance Engineer will then work together with the driver in a data feedback loop to correlate the data from the simulations to the car on track. As the driver increases his pace, the engineers will work with them to try to replicate the ideal laps from the simulation, considering the driver's individual style of driving.
5. Once the car comes off track, the telemetry from the car is returned to the headquarters of the F1 team to be used for future correlation of the strategy simulator and driver-in-loop driven simulator.
6. The Driver-in-Loop (D.I.L) simulator is used in conjunction with the Simulator Engineer and the Development Simulator Driver. The main aim of the Driver-in-Loop simulator is to correlate data and try new concepts that may not be generated in the main simulation. The D.I.L. simulator can also be used to account for human factors and mistakes that the main simulation cannot. This data is then re-circulated back into the main simulation as training data.

Having researched the typical use case of a simulation tool in motorsport, elements of this research can be used to inform the development of this project. Primarily, it is obvious that a lot of personnel will use this software and the data from it regularly. This necessitates an easy-to-use interface and interactions, alongside easy to read useable data. The user case also suggests that data should be easily transferable and easy to export from the simulation.

3.3 Analytical Work

The plan for this project revolved around building a solid foundation in the first few sprints of development so that by the time the Christmas break approach there was a solid platform to build the more complex features on top of. There was some time spent looking at previously developed tools and how the history of strategy software has gone, as discussed in Chapter 2.

It was discovered that the tools did not feature an in-built editing tool to create or adjust tracks on the fly. Developers would normally control these, and there seems to be a gap in the market here.

Therefore, the initial plan would be to build a working software which would allow a user to overlay a track map, pick key waypoints, which the software would then wrap a spline around and generate n number of points. This would be the fundamental data that would, in the future, be given width and have speed and telemetry calculations performed on it.

With this adaptability and flexibility built in from the start, physics could then be introduced to figure out corner radius, max velocities, acceleration, deceleration, and lap time. This would then be a solid platform to add features such as a degrading tyre model, multiple laps, stint lengths, and running full grand prix length distances with pit stops.

3.4 Architectural Level

Figure 3.2 shows how the code creates the framework for the simulation to create the lap time. This was extended to generate genetic lap times. The aim was to make the structure flow through a method to collect all the track's data and bring that data through to a common main method from which identical processes can be conducted. This supports the modular framework aim of the code and allows tracks to be added and removed from the program seamlessly. From the main method, a method can be called to generate the splines and break them into a set number of points. Following this, the physics methods are called, which generate the radius and max speed of each point around the track. These speeds are then fed into the telemetry method, which will then fetch the performance values of the car (i.e. the acceleration and braking ability) and the tyre parameters (i.e. the grip factor for each compound). A rudimentary method will then calculate the Lap Time of the three tracks, like Scalextric slot cars. ("The History of Scalextric," 2026).

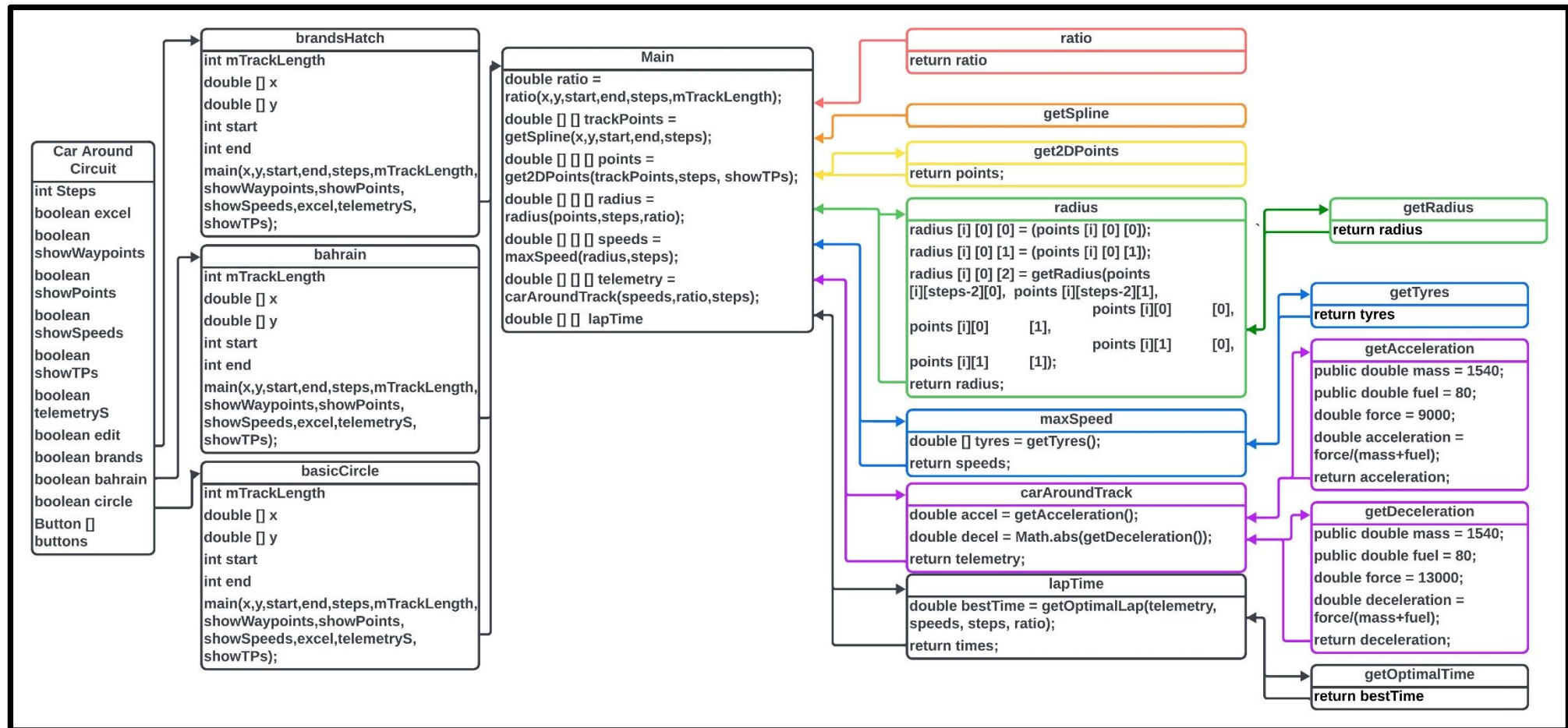


Figure 3.2 Breakdown plan of the simulator's possible landscape

3.5 Software Interactions

Considering that the project's UI/UX is full of compromises. The key to the project is a car around a circuit, and the ability to see the tracks being created and where the points are positioned along the splines is key to understanding how lap times are calculated. However, the most insightful data in motorsport typically comes from graphs and data points rather than from illustrations. Therefore, the decision was made to split the data representation between the UI/UX and an exported .xlsx Excel spreadsheet, which is like real race telemetry.

3.5.1 User Interface/Experience (UI/UX)

Within the software, the key focus is to represent the map of the given circuit, the points at which the car crosses points along the splines, and to give the user the ability to edit the track parameters, Genetic Algorithm (GA) settings, stint lengths desired and basic representation of lap/stint times. It was also decided to add functionality to rebuild the chosen parameters and to automatically launch the Excel sheet from the UI/UX. Fundamentally, the bright colours and customisation of the data being shown are intended to break down barriers to entry of motorsport understanding and allow the user to analyse how lap time is achieved around a track. An example of this is illustrated in **(Figure 3.3)**.



Figure 3.3 The UI of the Simulator while running

3.5.2 Excel

The development of this code was initially considered to be based on an AI-generated framework, like how the Genetic Algorithm was developed. However, it became apparent that a scratch-built Excel output using Apache POI would be more flexible and controllable. (Apache POI, 2026). The program allows data from the project. to be put into custom-made columns and rows. The library can then create charts from that data.

The Excel sheets provide an extension of the project equivalent to a telemetry tool in the real world. Inside the first sheet, all the raw data for the runs around the circuit are stored. This is the data that is then used by consecutive sheets.

Within the second sheet, there is a comparison of the speeds achieved around the three fundamental laps. This provides an insight into where each line is better around the circuit layout and gives an insight as to where a GA may choose to place itself around the circuit.

The next sheet is intended to compare the max speeds achieved between the GA that seeks the shortest distance it can, versus the GA that seeks the shortest time it can. This was illustrated to show the importance of conservation of momentum around a track and how, by seeking out the shortest distance possible, you can compromise sequences of corners and momentum, resulting in a slower lap time.

The final permanent sheet is a sequential addition to the previous chart, now adding in the central line of the track to draw a comparison of how the three separate approaches of sticking to a line, shortest path and conservation of momentum contrast each other.

Alternatively, when stint calculation is enabled, a subsequent sheet is created which allows the user to see data over the laps of the stint selected. It shows raw data, a chart showing lap times over the stint, and total stint time over the stint. An example of the Excel sheets is illustrated in **Figures 3.4 & 3.5**.

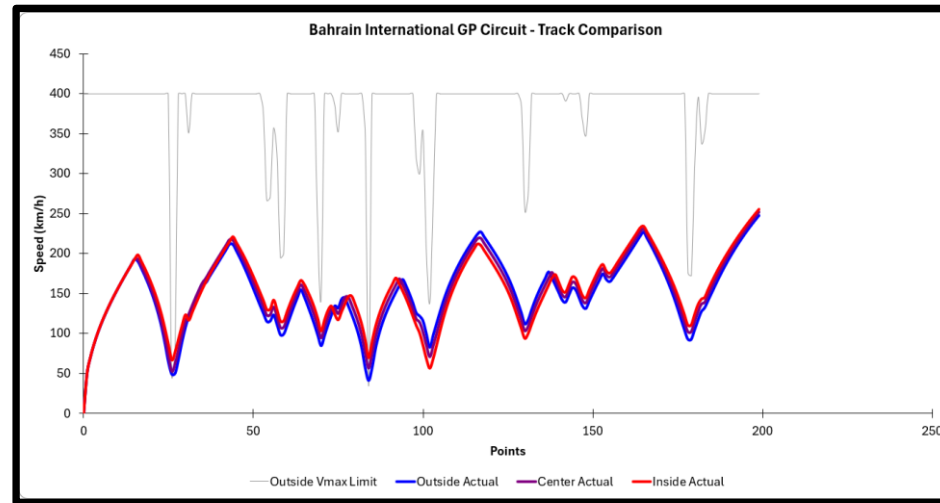


Figure 3.4 The Excel telemetry traces of the 3 original lanes vs. Max Velocity for the outside Lane at Brands Hatch

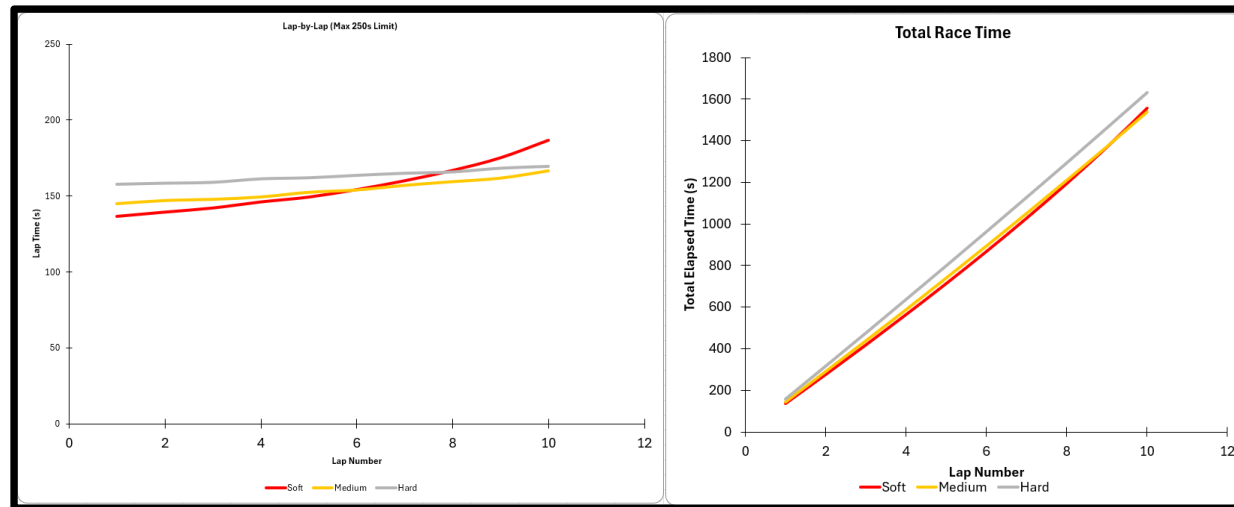


Figure 3.5 The Excel telemetry traces for a 30-lap stint of Brands Hatch Indy

3.6 Track Description

When choosing the desired tracks for this project, the main considerations included minimal elevation change; this ruled out tracks such as Spa-Francorchamps. A consecutive increase in complexity from the starting point of a circle was also desirable; the Brands Hatch Indy Layout stood out for its basic starting circuit. Despite the elevated drop in turn one, its relative uniformity, short length, and stable elevation alongside the plethora of data available online made it perfect for this simulation. The Bahrain GP Circuit became the next reasonable step up for its similar features. It is longer than Brands Hatch, has much higher top speed zones, more complex corners, especially through sector two, and has a low enough elevation change to make lap times relevant.

3.7 Spline

The spline library brings lots of functionality to our program and allows us to create the desired shapes of the circuits and their racing lines. The process of creating a spline can be explained as linking multiple cubic functions together to make one continuous shape. The spline libraries allow the definition of points to create the shapes of the tracks simply and continuously through a reusable method, which makes it interchangeable and modular.

3.8 Lap Times and Physics

The lap times and physics of this project are fundamental to the functionality, feasibility and credibility of the finished software and are built to generate data which the rest of the project relies on. Using the points of the Spline generated previously, the software works out the radius of the curve through a point at gate n . The curve begins at $n-1$, through n and runs to $n+1$ and marks that as the radius of the point in using the circumcentre equations (2.6),(2.7)&(2.8) and then the distance equation (2.9). See figure 3.6.

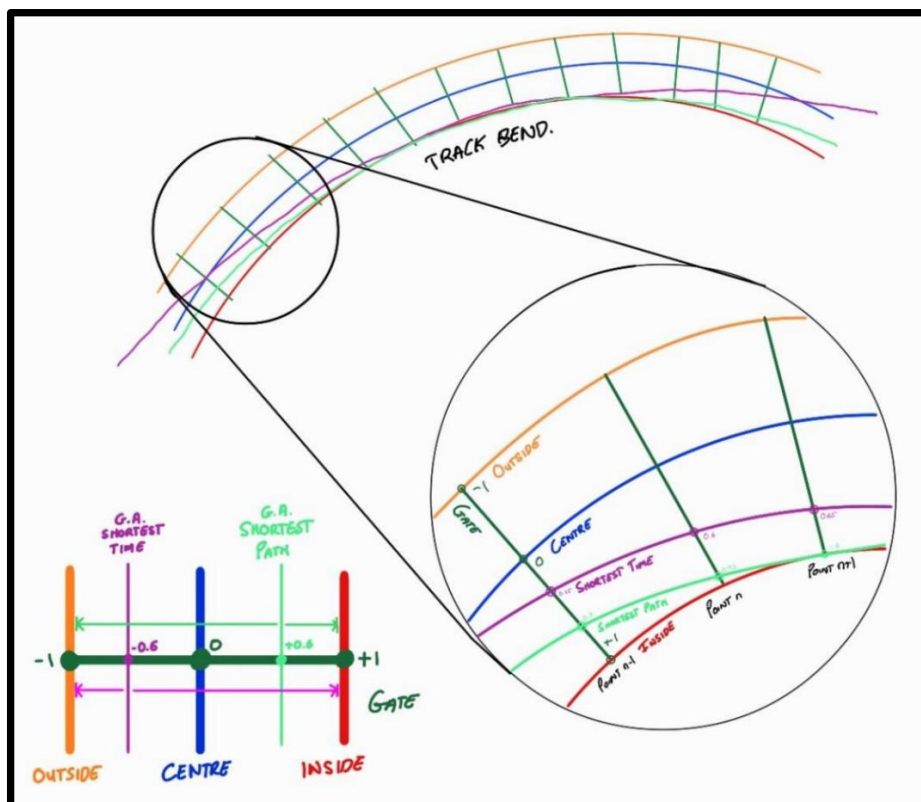


Figure 3.6 The make-up of the simulation and path bounds (GA or otherwise)

Using the radius of each point, the next method then gets the tyre data of the car and uses the max speed around a corner equation (2.10) to get a maximum velocity (**vMax**) figure for each point around the circuit.

The telemetry method then gets the maximum achievable speeds around the track by the car by using the three kinematic equations (2.1),(2.2)&(2.3), the distance equation (2.9) and the car's acceleration and deceleration figures. The method runs a forward loop in which it accelerates as much as possible, and if the velocity of the car exceeds vMax, it instantly decelerates to vMax. Subsequently, the method then runs backwards and at each point decelerates from vMax to find the point at which the car must brake to be at or below vMax at that point. This logic then provides us with a very recognisable lap telemetry to what is seen in modern motorsport strategy tools referenced in the review of existing platforms (Ch2.2).

Finally, the lap time method takes all these velocity figures from around the track and uses them to calculate the lap time using the distance formulae (2.9) and the speed, distance, and time equation (2.13). This lap time then represents the time taken to get around that specific circuit, with that specific tyre, with the given car characteristics, within the bounds of calculated and quantified physics.

3.9 Addition of Genetic Algorithm to find race line

Once the car had completed a track on the three existing circuits, the objective became to create a more sophisticated lap that would choose a place at point n between the inner and outer line, which would be used to build its own line around the track. The constraints of time had begun to take hold in the project, and thus it was decided that an element of Large Language Model (LLM) coding could be utilised to provide a framework to build the GA on.

3.9.1 Description

When deciding how to approach the GA, the decision was made to provide it with the points that have been created for the three default tracks, how the radius, vMax and telemetry etc was created, alongside how many generations, population and elites would be utilised. The prompt would also need to be provided with a basic structure and what its main method would need to return to the program.

3.9.2 AI and Prompts

Once the prompt at (A2.1.1) was fed into ChatGPT 5.2 and a basic structure was returned, which had identical versions of the physics methods, its own calling method, and the necessary GA methods, such as mutate and assortByFitness. This was advantageous because it provided a place that could be built on for the project's needs.

3.9.3 Building on Framework

Once the method was introduced, it needed some debugging; the LLM had changed some variables' names and had also redeclared variables, not realising they had been publicly declared elsewhere. From this point, time was taken to verify and understand what had been created and how it worked. Some changes were made to the mutation figures to make it more aggressive to changes, and once integrated into the rest of the software, reliable results were anticipated.

3.9.4 Integration

Once fully integrated and working, the data returned from the GA was hopeful. A pre-existing “Brute Force” method made earlier to try to find the shortest path around the track had less-than-ideal results. This was modified to become a GA focusing on the shortest distance rather than time, using the template from the new GA. Tests were set up to ensure physics methods in the GA match those existing in the original program, and also tests to ensure points were translated properly, along with assurances that both GA algorithms were improvements compared to basic lines. The completed GA algorithm is referenced in (A3.1).

3.9.4 Final Product

When the Genetic Algorithm is run, it generates a point within each gate between -1 and 1. These points become strung together by a spline. These points are then evaluated in the same fashion as described in 3.8. See Figure 3.6 for further explanation on the implementation of the paths from the Genetic Algorithm.

3.10 Race Stints and Tyre Degradation

Finally, as an add-on to the entire project, there was an extra method added to allow for a stint to be requested for the user, which would run a stint of n laps across three compounds of tyres while degrading the tyre performance over completed laps. The data being fed to the Excel sheet suggested that the GA was adapting to decreased tyre grip and stint lengths, and lap times from the stint were positive and realistic to what would be expected from the real car. This confirmed that with expansion, the framework created in this project is capable (with the help of another GA) to calculate and recommend race strategies based on car and tyre performance for a flexible race distance. The output of this to Excel is available at (Figure 3.5).

3.11 Statistical Data

To test the performance of the Genetic Algorithm, the simulation has a method which simulates a set number of laps, producing the genetic best lap time it can. These lap times are then recorded, and from these times, the Mean Average, Standard Deviation, and Coefficient of variation are calculated using the equations defined in chapter 2 (equation 2.16, 2.17 & 2.18). See Figure 3.7

```
[17:00:40] 20 laps per tyre, generations: 2000, population: 150, elites: 15
Tyre      Mean Lap      TimeStandard Deviation      Coefficient of Variation
Soft: Mean: 50.565      SD: 0.228      CV: 0.450%
Medium: Mean: 54.572      SD: 0.361      CV: 0.661%
Hard: Mean: 60.153      SD: 0.313      CV: 0.521%
```

Figure 3.7 Statistical Data figures outputted to Console.

Chapter 4: Evaluation

In this chapter, the discussion covers the code created, how it is evaluated and tested to ensure it is credible, reliable and accurate.

4.1 Metrics of Success

To critically assess our model, which does not account for aerodynamic grip or the ability to lose traction, we need to find a suitable real-life comparison with low amounts of aerodynamic drag, good straight-line speed, and the ability to be equipped with racing slick tyres. The most suitable car for these requirements is the BMW M4 GT4 EVO. ("BMW M4 GT4 EVO," 2026). The car is not in the highest class of GT racing, has low to moderate levels of downforce, and has competed on the two chosen tracks, Bahrain International Circuit and Brands Hatch Indy Circuit. The three parameters taken from the car are acceleration, braking and mass, of which mass includes fuel weight.

The assessment of this project will consist of drawing comparisons to real-life data and video evidence that can be compared to the data generated.

4.2 Comparison with Circle Solution

The definitive evaluation throughout the design of the software has been the comparison to a completely circular track. This circular track is used as a way of verifying the physics, as any of the calculations of velocity or radius are inherently easier on a circle and verification calculations are easy to complete.

In Figure 4.1, the manual calculations are compared to the output from the simulation. The simulator produced a mean max Velocity of 21.57, whereas the calculations produced a VMax of 17.2 m/s. This difference of approximately 4.37 m/s can be explained by the design of the simulator.

In the design of the simulator as stated in the previous chapters the tracks are defined by splines. These splines are made up of a combination of cubic curves meaning that the "circle" track is never a perfect circle. Factors such as local curvature, where a radius is never constant for a spline, discrete sampling, where the definition (number of points in the circle) of the simulation is not high enough and possible small discrepancy in the code make the difference bigger.

Circle max Velocity Calculations:

$$\text{Circle Radius } (r) = 300 \text{ pixels}, \frac{300}{13.93} \text{ m}, \quad \text{ratio} = 13.93/1$$

point 1 = (500,400), point 2 = (800,100), point 3 = (1100,400), μ of soft tyre = 1.4

Calculate Centroid:

(From 2.4.4). $\text{determinant}(d) = 2(x_1(y_2 - y_3) + x_2(y_3 - y_1) + x_3(y_1 - y_2))$

$$(d) = 2(500(100 - 400) + 800(400 - 400) + 1100(400 - 100)) = 360,000$$

$$\text{(From 2.4.5)} \quad c_x = \frac{(x_1^2 + y_1^2)(y_2 - y_3) + (x_2^2 + y_2^2)(y_3 - y_1) + (x_3^2 + y_3^2)(y_1 - y_2)}{d}$$

$$c_x = \frac{(500^2 + 400^2)(100 - 400) + (800^2 + 100^2)(400 - 400) + (1100^2 + 400^2)(400 - 100)}{360,000}$$

$$c_y = \frac{(x_1^2 + y_1^2)(x_3 - x_2) + (x_2^2 + y_2^2)(x_1 - x_3) + (x_3^2 + y_3^2)(x_2 - x_1)}{d}$$

$$c_y = \frac{(500^2 + 400^2)(1100 - 800) + (800^2 + 100^2)(500 - 1100) + (1100^2 + 400^2)(800 - 500)}{360,000}$$

$$c_x = 800, c_y = 400$$

Calculate radius:

(from 2.4.6) $s = \sqrt{(x_i - x_{i-1})^2 + (y_i - y_{i-1})^2}$

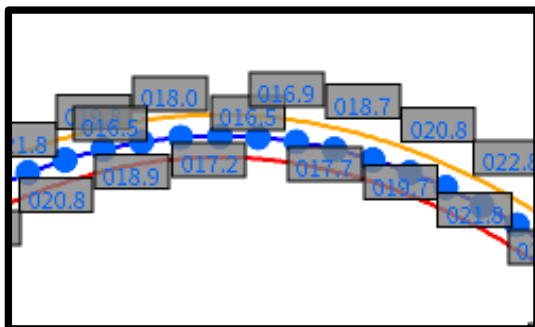
$$r = \sqrt{(c_x - x_1)^2 + (c_y - y_1)^2}$$

$$r = \sqrt{(800 - 500)^2 + (400 - 400)^2} = 300 \text{ pixels}$$

Calculate max Velocity:

(from 2.4.7) $v = \sqrt{\mu \cdot g \cdot r}$

$$v = \sqrt{(1.4) \cdot (9.81) \cdot \frac{300}{13.93}} = 17.2 \text{ m/s}$$



Simulation circle with 100 points

Figure 4.1 Car around a circle and 4.2 calculation

4.3 Comparison with Real Tracks

Once the physics of the project has been verified, the project focus shifts to the tracks. Once a scratch-built editing mode is enabled, an overlay image of the circuit is drawn at the back of the UI and by switching on waypoints, you can then see the waypoints that the project draws from. These points can then be modified manually via the code for this version of the project; however, future developments are discussed in (5.4) surrounding waypoint adjustments from the UI.

The lines shown on Open-Racer.com provide a good idea of what a good racing line at these circuits is and provide good comparison shots at Brands Hatch between the two in satellite view against our software (**Figure 4.1**); and similarly for Bahrain in (**Figure 4.2**) (Open-Racer.com, 2026a, 2026b).

Videos of laps are posted on YouTube as a way of comparing how the car reacts to certain corners and how it accelerates and brakes. This served as further assurance that the simulation is synced with the performance of the BMW; however, the main goal of the code was to be adaptable, and through the acceleration and braking performance parameters and flexibility of the tyre model, the code can be adapted to any non-aero car. (Nick Darlow, 2017) (Virtual Reality Racing Club, 2025).

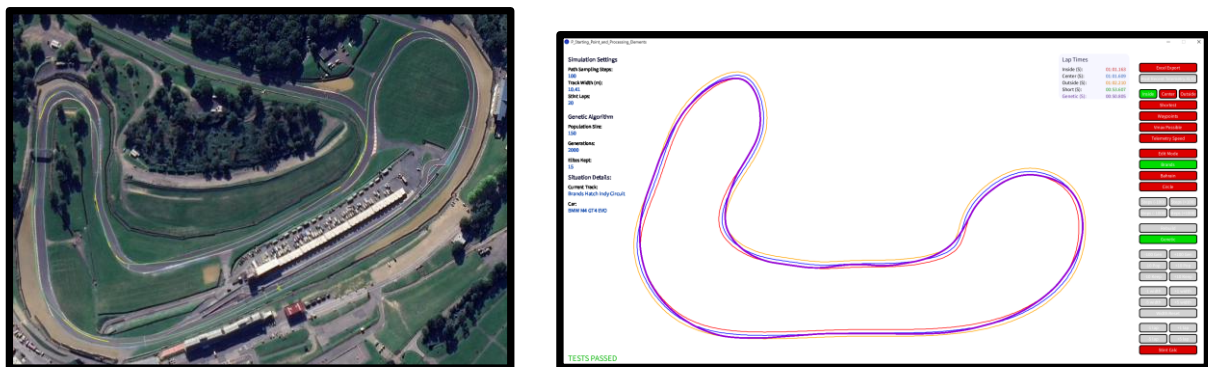


Fig 4.2 Brands Hatch Indy Open-Racer compared to the simulator (Open-Racer.com, 2026a)

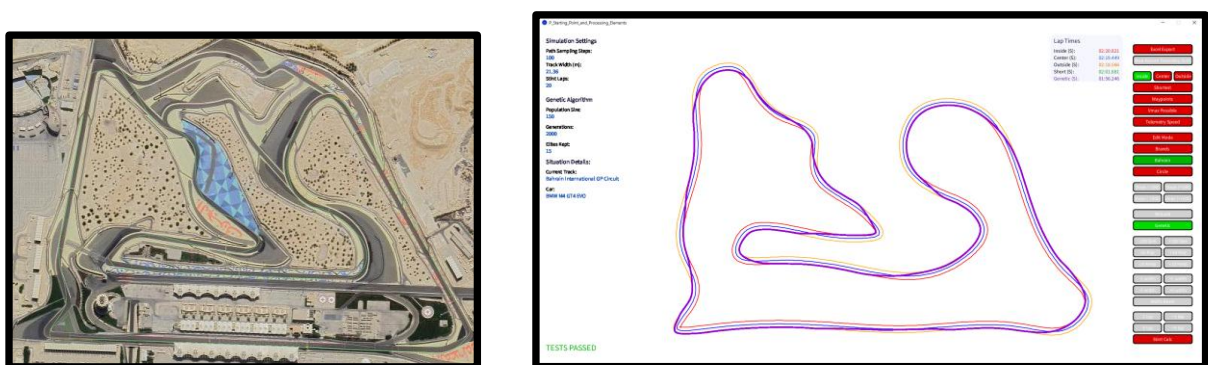


Figure 4.3 Bahrain GP Circuit Open Racer compared to Simulator.(Open-Racer.com, 2026b)

4.4 Comparison with Real Lap Times

Building on the previous chapter, the final lap times being output by the software are compared to those done both in reality and in simulators. In Bahrain in 2018, the BMW M4 GT4 competed in the GT4 International Cup and had a best time of 2:12:242 compared with the simulation lap time of 1:55.440 on hard tyres over 20 laps with population size 150, 2000 generations and 15 elites being kept. ("GT4 International Cup 2018 - Qualifying Results - Racing Sports Cars," 2018). Meanwhile, in Brands Hatch, an M4 GT4 completed the Indy circuit with a time of 1:01.51 compared to the simulation with the same parameters doing a 1:00.161 on racing softs (Nick Darlow, 2017). With the tools in the simulator, further correlation can be achieved by changing mass, fuel, tyre, acceleration, deceleration, and, in the future, temperature and wind direction. These factors will allow the time prediction to become more accurate and to identify in what areas the simulations are being produced inaccurately, alongside the scenarios in which the driver is not getting the full potential out of the car. The performance of the GA is further discussed in 4.5.7.

4.5 Software Verification and Testing

In the following sub-sections, the testing and verification of the software are discussed at length.

4.5.1 Testing and Verification Approach

The testing approach for this project revolves around the physics of the project and verifying that it is mathematically correct, leading to an accurate, adjustable method that can be correlated with any non-downforce car at this stage. The data produced by any simulation is credible because of its accuracy, and a large focus during the development of the software was on test-driven development, in which the tests would be developed first and the methods to produce desired results second.

4.5.2 Circle Verification

Throughout the project, the Circle track has been a means of verifying the outputs of the physics and GAs, as it displays the racing lines achieved in the UI, which can be quickly assessed and checked mathematically.

4.5.3 Checking Points Test

The Checking Points test within the Testing.pde is designed to check across the arrays that carry the points, vMax, radius and speed data to ensure that the x and y co-ordinates that are transferred between them stay accurate between iterations. If this is not the case, it is flagged both in the console and in the UI.

4.5.4 Checking Radius Calculations Test

The Checking Radius Calculations test is designed to take in all the radius calculations done within the software and check them against the mathematical calculation of the radius. This ensures that the mathematics has been executed correctly, but also that all variables have been assigned correctly to their correct slot in the array. If this is not the case, it is flagged both in the console and in the UI.

4.5.5 Checking Speed Calculations Test

The Checking Speed Calculations test is designed similarly to the previous test on the radius, and its main job is to evaluate and verify the results of the max Speed Calculations. It takes in the radius array, runs through each parameter, and checks it

against the result present in the speeds array. This ensures that the mathematics have been executed correctly, but also that all variables have been assigned correctly to their slot in the array. If this is not the case, it is flagged both in the console and in the UI.

4.5.6 Checking Lap Time Progression Test

The checking lap time progression test is designed to simply ensure that as the simulation gets more intelligent, the lap times improve. Time should improve when racing lines are represented using GA rather than the preliminary use of Scalextric style. Times should also improve when moving to a genetic algorithm designed to find the fastest time and conserve momentum. This test works on the logic of:

$$\textit{Time of original inside} < \textit{Time of Short Lap} < \textit{Time of Shortest Time Lap}$$

Although this may be accurate, it indicates that the data may need to have extra verification. If this is detected, it is flagged both in the console and in the UI.

4.5.7 Checking Mean and Standard Deviation of Genetic Algorithm

The main method of testing the Genetic Algorithm was to use the method described in 3.11. This method gives us the mean, standard deviation and coefficient of variance for a G/A output for a given tyre over a set run of laps. What we can see from the data is that tyres with a higher grip coefficient have less variance than those with less, as the tyre is more predictable and it is easier to stay on a set line. Also, with increasing laps, the standard deviation increases as the likelihood of outliers increases and the data spread increases. This supports the genetic algorithm and shows its running well. (Tulabandhula and Rudin, 2014)

4.5.8 Checking against existing data

Finally, throughout development, the last step in verification of data is to check times and lines used against those that exist in reality and DIL simulators. The ability to identify the data coming from the program as realistic and as identifiable is a key part of the process and allows the programmer to examine the real-life tangible goals of the program.

4.5.9 Interpretation of the results

When assessing the results of the tests above, it can be concluded that they provide a basis to assign accuracy to the results that they produce. Throughout the tests, we can verify that the x,y points used throughout the process are accurate as they move through the simulation; that those points produce accurate radius calculations. Then, from the radius, produce accurate vMax profiles, providing a reliable platform for lap times to be produced and genetic algorithms to use as a framework.

Chapter 5: Conclusion

In this chapter, the achievements of the project are discussed. It includes a project reflection and then concludes with some suggestions for future work.

5.1 Achievements

The achievements of the project are as follows:

The project led to the development of a physics model based on friction and centripetal force that provided the maximum speed at which a curve can be traversed.

A data structure was developed to map a track, utilise the physics model, and show the basic paths that a car could take.

Further Code was developed to measure the time taken for a given path to allow lap times to be produced.

Two genetic algorithm types were developed to get two racing lines: one prioritising the shortest distance and one prioritising the shortest time. These were then compared to show the contrast of approaches.

Two final methods were created. The first was used to run a car over a stint using the shortest time GA, while reducing tyre grip and fuel over consecutive laps. The second was used to generate several laps and get the mean, average, standard deviation and coefficient of variance.

To interpret all this data, a UI/UX package was created using processing, which allows the user to view lines, get lap times, adjust GAs, create custom stints and switch between tracks instantly. This and other features in the UI/UX allow the program to be an approachable learning tool for motorsport fans, engineers, and aspiring physicists.

5.2 Project Reflection

Reflecting on the project as a journey, the objectives as outlined in chapter 1, were to take inspiration from the world of motorsport, and particularly people like Bernie Collins, and to test the statement “There are quite a few things that are not easy to model... A lot of what you do is estimation.” (Collins, 2025). To verify what Collins said, it is not easy to model motorsport strategy and the characteristics of a car; however, estimation is possible, and in the high stakes of motorsport innovation, it is crucial.

This project aimed to be a credible, accurate and approachable framework with the potential to be expanded upon. The makeup of the code is modular, well-defined and can be followed. Tracks can be created, read, updated, and deleted without impacting the wider program, and data can be extracted and data extracted quickly. Testing is effective and seeks to target where obvious flaws may exist, allowing for confidence in results. Exported data is well structured, considered heavily and aims to give the most insightful data first.

Future expansion of this project would be beneficial, and it would have potential as a stepping stone to the industry, a basis for a strategy-based game, or to be transferred to a more capable and competitive language base to be expanded as the framework for a much bigger tool.

5.3 Future Work

Ambitions for this project going forward could bring the software in many different directions.

5.3.1 Framework and Improved Physics

Firstly, regardless of where this software goes beyond the conclusion of the final year project element, expansion and integration of a more sophisticated physics model to unlock the car's potential to brake traction, slide, and even break the bounds of the track if overly ambitious, would add a completely new element. It would allow for a much more accurate view of the tyre model and how sliding would degrade the tyre beyond what is possible in this version. This would provide an ideal framework to be ported to a more capable language and expanded to be a credible tool within the motorsport space.

5.3.2 Stand-Alone Strategy video game

Over the last few years, especially with the increasing popularity of motorsport, an interest in motorsport strategy games has emerged. This tool could be the backbone for a much more advanced strategy game in which the player would have access to similar charts to real life. It would compete with games such as F1 Manager. ("F1® Manager 2024," 2025) and would be liked by fans.

5.3.3 Integration with a simulation game

Within the sim racing community, tools such as Sim Racing Telemetry ("SRT | Sim Racing Telemetry," 2026) are becoming increasingly popular, and with the lack of a free simulation solution for sim racers, this tool could be released as an open source project. If the software was linked to race simulators and video games, it could analyse telemetry and use it as training data for a more advanced Genetic Algorithm, to give ideal racing lines and stint predictions to the user. The passion within the space exists to bring this application forward and it would be a considerable way to give back to the motorsport community.

Bibliography

- A brief history of computing in Formula 1 [WWW Document], 2016. URL <https://www.mclaren.com/racing/team/a-brief-history-of-computing-in-f1-1052199/> (accessed 2.10.26).
- Analysing Racing Lines in Telemetry Data [Sim Racing Telemetry Docs] [WWW Document], 2026. URL <https://docs.simracingtelemetry.com/kb/how-to-analyze-racing-lines> (accessed 2.16.26).
- Apache POI, 2026. Overview (POI API Documentation) [WWW Document]. URL <https://poi.apache.org/apidocs/5.0/> (accessed 3.12.26).
- Balci, O., 1995. Principles and techniques of simulation validation, verification, and testing, in: Proceedings of the 27th Conference on Winter Simulation, WSC '95. IEEE Computer Society, USA, pp. 147–154. <https://doi.org/10.1145/224401.224456>
- Bloch, J., 2008. Effective Java. Addison-Wesley Professional.
- BMW M4 GT4 EVO [WWW Document], 2026. URL <https://www.bmw-m.com/en/fastlane/motorsport/race-cars/bmw-m4-gt4-evo.html> (accessed 2.10.26).
- Collins, B., 2025. How to Win a Grand Prix: From Pit Lane to Podium, the Inside Track on F1, illustrated ed. Hachette Australia, 2024.
- Collins, B., 2023. F1 Strategy: How Formula 1 Teams Optimize Analysis and Racing. Catapult. URL <https://www.catapult.com/blog/formula-1-race-strategy-analysis> (accessed 2.16.26).
- F1® Manager 2024 [WWW Document], 2025. . F1® Manag. 2024. URL <https://www.f1manager.com/> (accessed 3.4.26).
- Fédération Internationale de l'Automobile, 2024. 2025 FORMULA ONE SPORTING REGULATIONS.
- Gaonkar, A., 2024. What Causes a Lock-Up in F1? [WWW Document]. The SportsRush. URL <https://thesportsrush.com/f1-news-what-causes-a-lock-up-in-f1/> (accessed 3.11.26).
- GT4 International Cup 2018 - Qualifying Results - Racing Sports Cars [WWW Document], 2018. URL <https://www.racingsportscars.com/results/qualifying/Bahrain-2018-12-02g.html> (accessed 3.4.26).
- Jamieson, C., 2020. Top Gear's guide to driving well: holding a powerslide [WWW Document]. Top Gear. URL <https://www.topgear.com/car-news/supercars/top-gears-guide-driving-well-holding-powerslide> (accessed 3.11.26).
- Lakhani, S., 2017. Trade Secrets and Industrial Espionage in Formula One Motorsports. Denver J. Int. Law Policy 46, 223–242.
- Lee, L., 2020. The Difference Between Productive Struggle and Spinning Your Wheels [WWW Document]. Edutopia. URL <https://www.edutopia.org/article/difference-between-productive-struggle-and-spinning-your-wheels/> (accessed 3.11.26).
- LLC, C., 2026a. Mean Calculator | Average Calculator [WWW Document]. CalculatorSoup. URL <https://www.calculatorsoup.com/calculators/statistics/average.php> (accessed 3.14.26).
- LLC, C., 2026b. Standard Deviation Calculator [WWW Document]. CalculatorSoup. URL <https://www.calculatorsoup.com/calculators/statistics/standard-deviation-calculator.php> (accessed 3.14.26).

- LLC, C., 2026c. Descriptive Statistics Calculator [WWW Document]. CalculatorSoup. URL <https://www.calculatorsoup.com/calculators/statistics/descriptivestatistics.php> (accessed 3.14.26).
- LLC, C., 2026d. Friction Calculator $f = \mu N$ [WWW Document]. CalculatorSoup. URL <https://www.calculatorsoup.com/calculators/physics/friction.php> (accessed 3.14.26).
- McLaren, 2013. F1 Framework: Simulating a lap. F1 Framew. URL <https://f1framework.blogspot.com/2013/02/simulating-lap.html> (accessed 2.10.26).
- Nick Darlow, 2017. Brands Hatch Indy circuit in a BMW M4.
- Open-Racer.com, 2026a. Racing Line for Brands Hatch - Ideal Acceleration and Braking Points [WWW Document]. Brands Hatch Circuit - U. K. Open Racer. URL <https://open-racer.com/brands-hatch/r/tciGPUuS5FTaLSbJVOQM> (accessed 3.4.26).
- Open-Racer.com, 2026b. Racing Line for Bahrain - Ideal Acceleration and Braking Points [WWW Document]. Bahrain Int. Circuit - Bahrain Open Racer. URL <https://open-racer.com/bahrain/r/D1oh7HhRDirlgspqdCTb> (accessed 3.4.26).
- O'Regan, D., 2000. Real World Physics. Folens Publishers.
- Processing Reference Page [WWW Document], 2025. URL <https://processing.org/reference>
- Race Analysis Software | Strategy & Circuit, 2026. . Catapult. URL <https://www.catapult.com/solutions/racewatch> (accessed 2.16.26).
- RaceWatch in F1: High-Tech Decision Engine Tested at 2025 Miami GP [WWW Document], 2025. Des. News. URL <https://www.designnews.com/automotive-engineering/racewatch-f1s-high-tech-decision-engine-faces-ferraris-miami-gp-dilemma> (accessed 3.10.26).
- Ross Brawn and Adam Parr, 2017. Total Competition: Lessons in Strategy from Formula One. Simon & Schuster UK.
- spark.iop.org, 2026. Proof of $F = mv^2/R$ | IOPSpark [WWW Document]. URL <https://spark.iop.org/proof-f-mv2r> (accessed 3.14.26).
- SRT | Sim Racing Telemetry: designed for pro and casual sim racers [WWW Document], 2026. URL <https://www.simracingtelemetry.com/> (accessed 2.16.26).
- The History of Scalextric [WWW Document], 2026. . Scalextric UK. URL <https://uk.scalextric.com/about-scalextric> (accessed 3.3.26).
- Tudor, E.S., 2019. The Emergence of eSport During Covid-19: How Sim Racing Replaced Live Motorsport in 2020 1.
- Tulabandhula, T., Rudin, C., 2014. Tire Changes, Fresh Air, and Yellow Flags: Challenges in Predictive Analytics for Professional Racing. Big Data 2, 97–112. <https://doi.org/10.1089/big.2014.0018>
- Virtual Reality Racing Club, 2025. BMW M4 GT4 Hot Lap Bahrain.
- Weisstein, E.W., 2026. Circumcenter [WWW Document]. URL <https://mathworld.wolfram.com/Circumcenter.html> (accessed 3.14.26).
- What is Torque? | Mountz [WWW Document], 2018. URL https://www.mountztorque.com/What-is-Torque_3?srsId=AfmBOorDwsxEbZ88sdK7ymb3Zq45XlcSkBzg7j8HXf0wbbxG4VhiLt1R (accessed 3.11.26).

whitman.edu, 2026. 1.2 Distance Between Two Points; Circles [WWW Document].
URL https://www.whitman.edu/mathematics/calculus_online/section01.02.html
(accessed 3.14.26).

GitHub Repository

USERS SEEN:

KL74IRE: Personal GitHub account evidence available on request

GLEN KELLY: Maynooth Git-hub account

LINK:

https://gitlab.cs.nuim.ie/u230341/glen_kelly_22443026_fina_year_project_2025.git

Appendices

Appendix 1: Imagery, Diagrams and Figures



Figure 2.1 ("RaceWatch in F1," 2025)



Figure 2.2 (Jamieson, 2020)

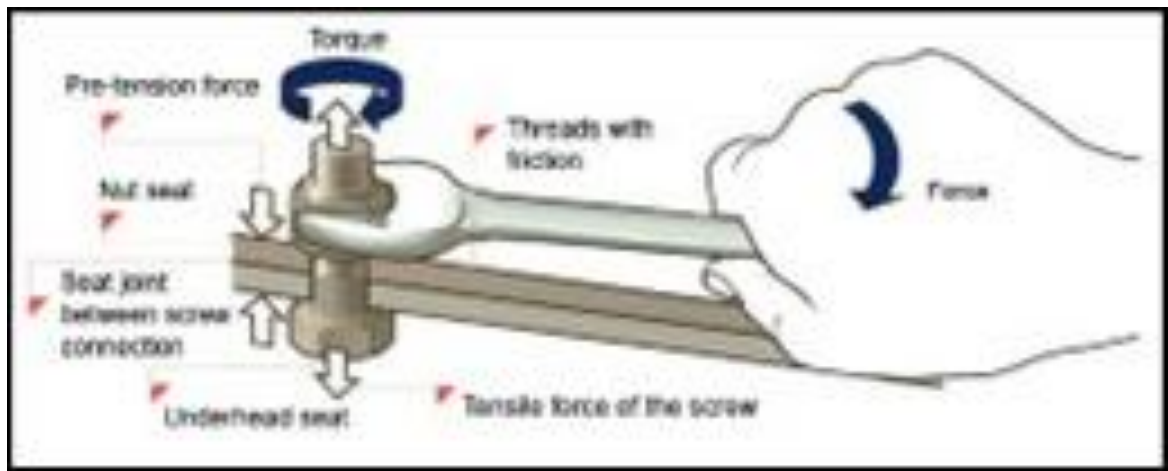


Figure 2.3 ("What is Torque? | Mountz," 2018)



Figure 2.4 (Lee, 2020)



Figure 2.5 (Gaonkar, 2024)

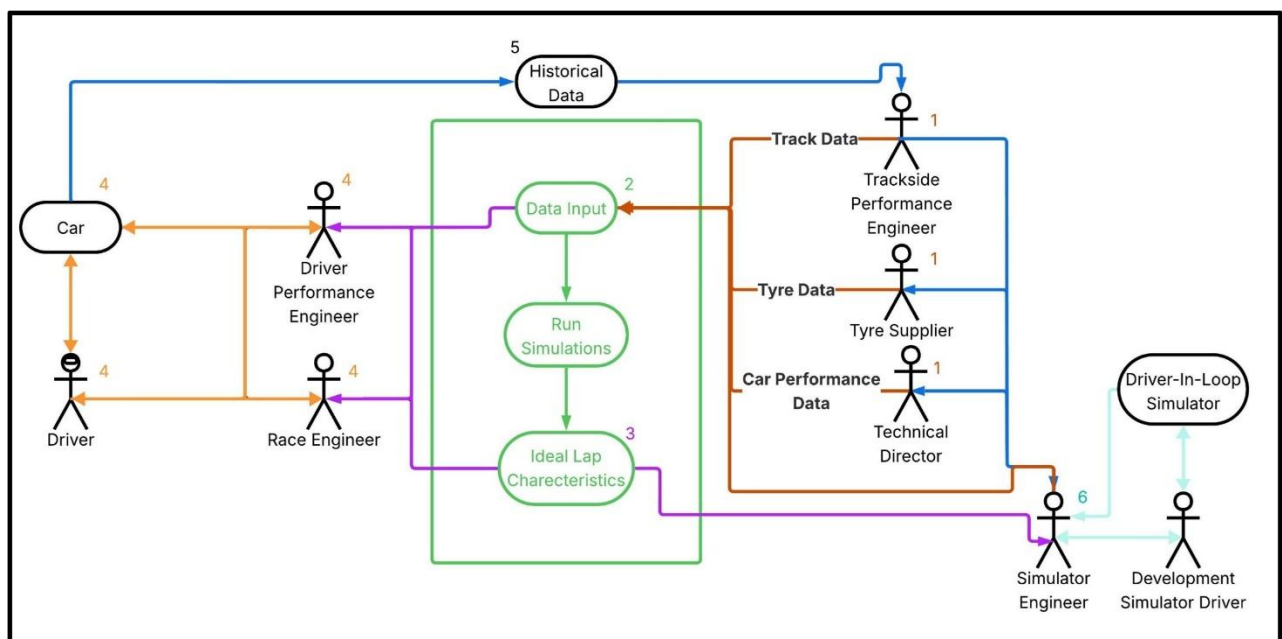


Figure 3.1 Use Case

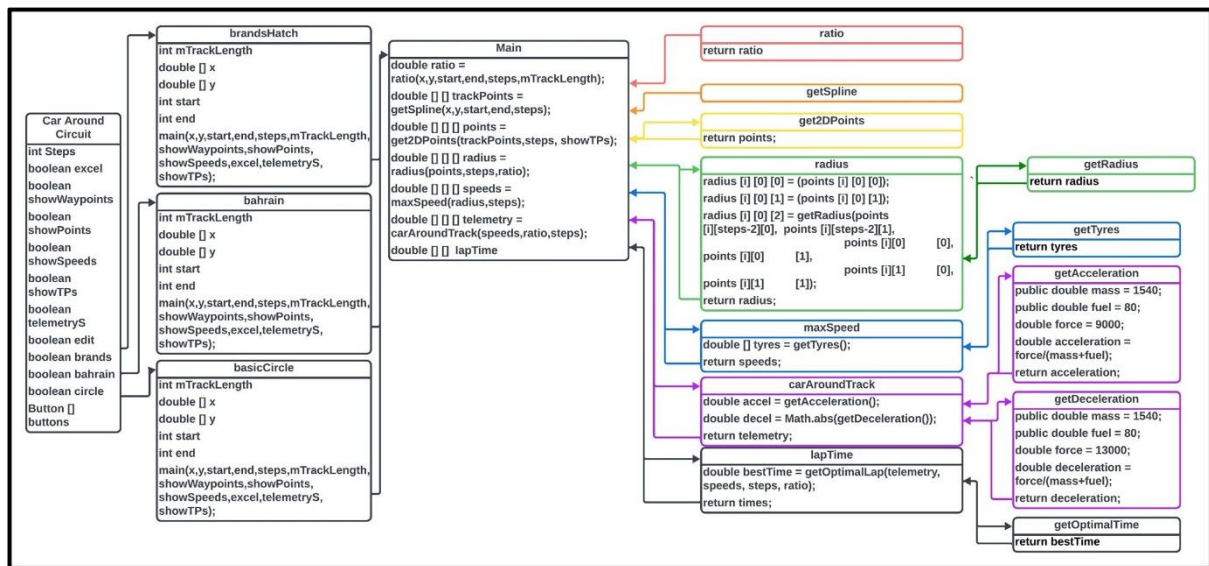


Figure 3.2 Breakdown in plan



Figure 3.3 Simulator UI

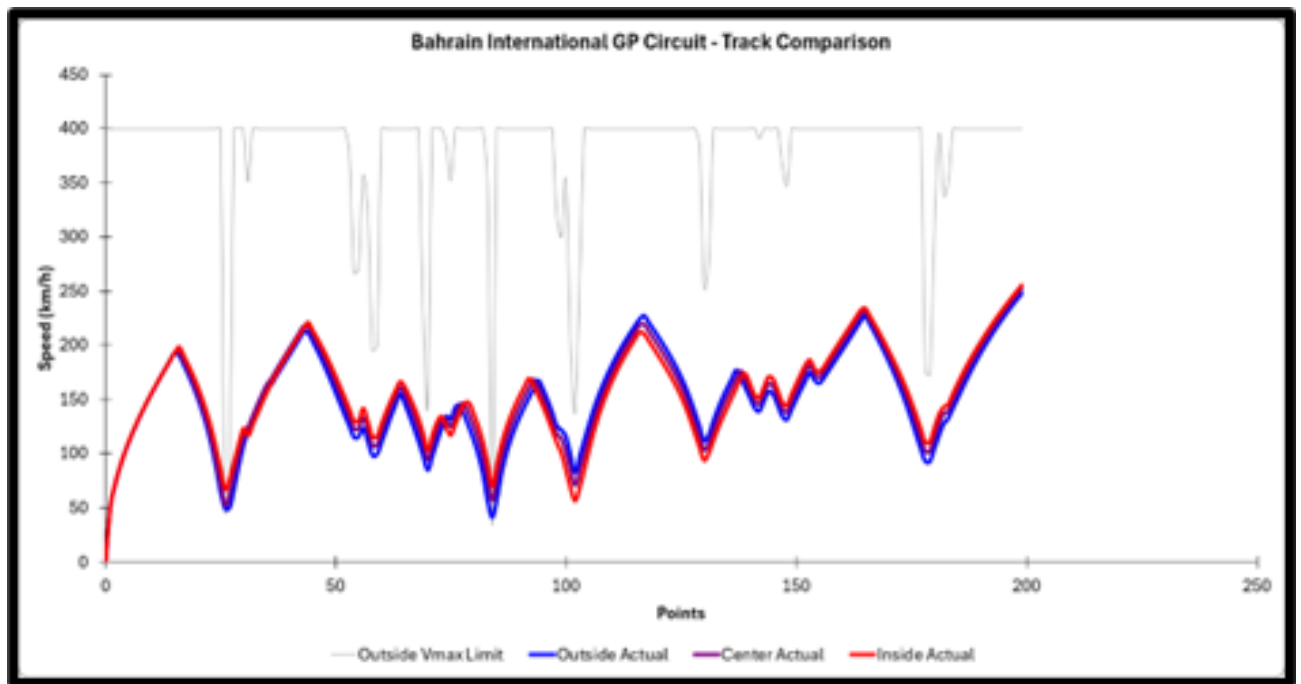


Figure 3.4 Telemetry

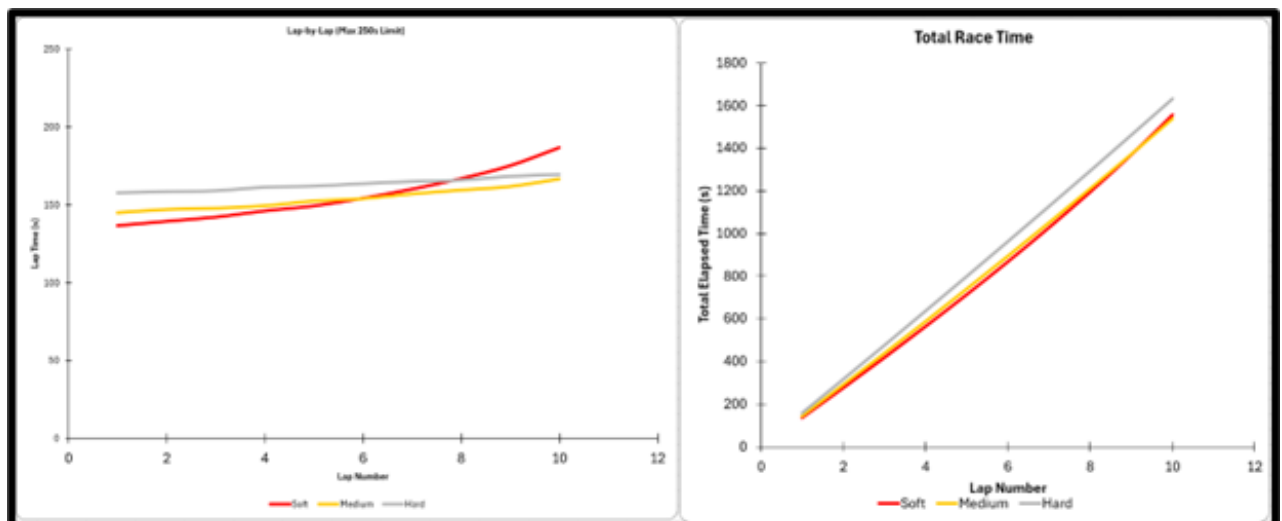


Figure 3.5 Stint Telemetry

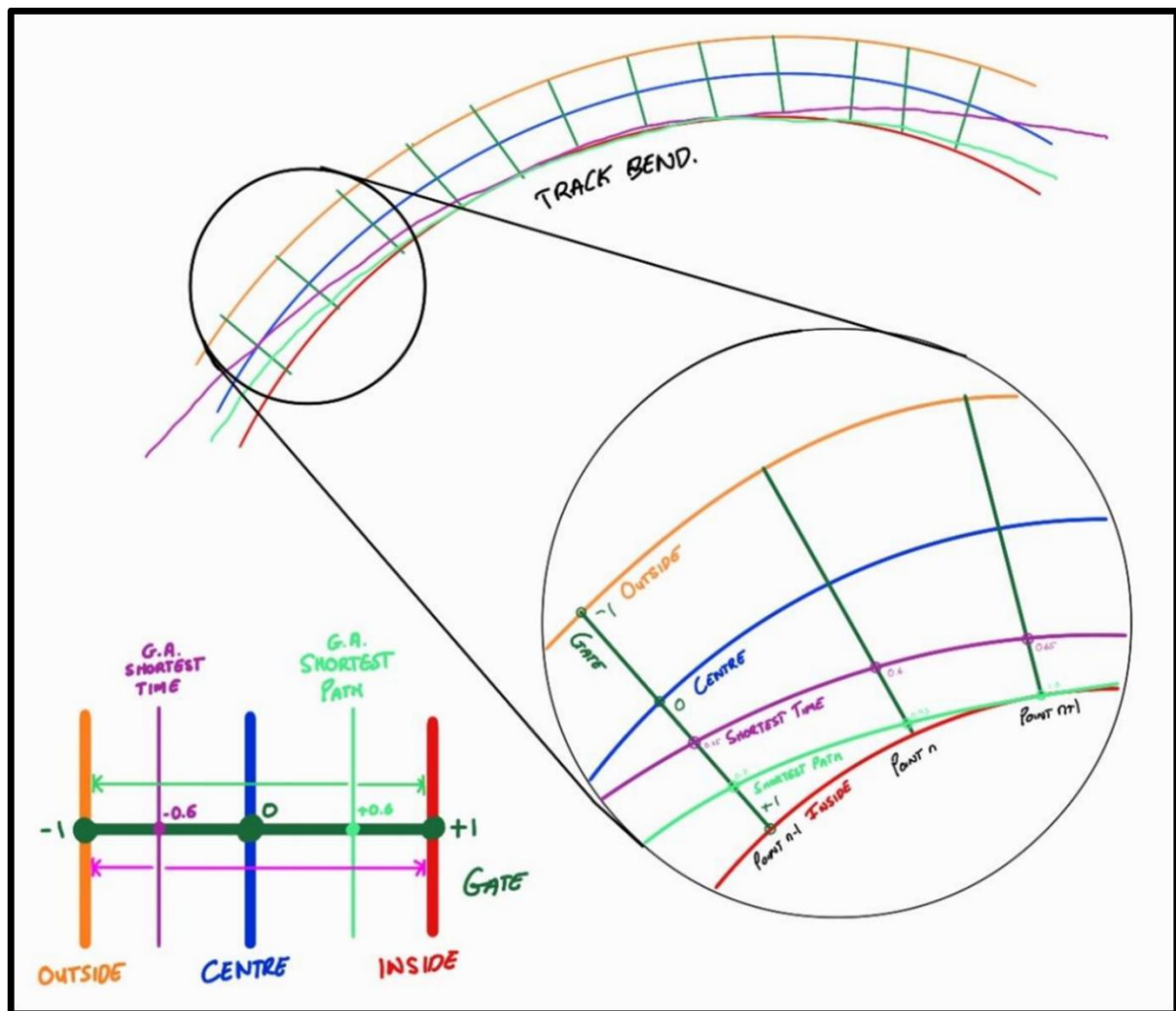


Figure 3.6 Gate Expansion

```
[17:00:40] 20 laps per tyre, generations: 2000, population: 150, elites: 15
Tyre      Mean Lap      TimeStandard Deviation      Coefficient of Variation
Soft: Mean: 50.565      SD: 0.228      CV: 0.450%
Medium: Mean: 54.572      SD: 0.361      CV: 0.661%
Hard: Mean: 60.153      SD: 0.313      CV: 0.521%
```

Figure 3.7 Stats Output

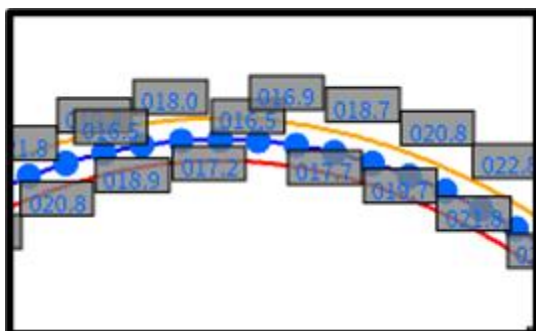


Figure 4.1 Circle Speed



Figure 4.2.1 (Open-Racer.com, 2026a)

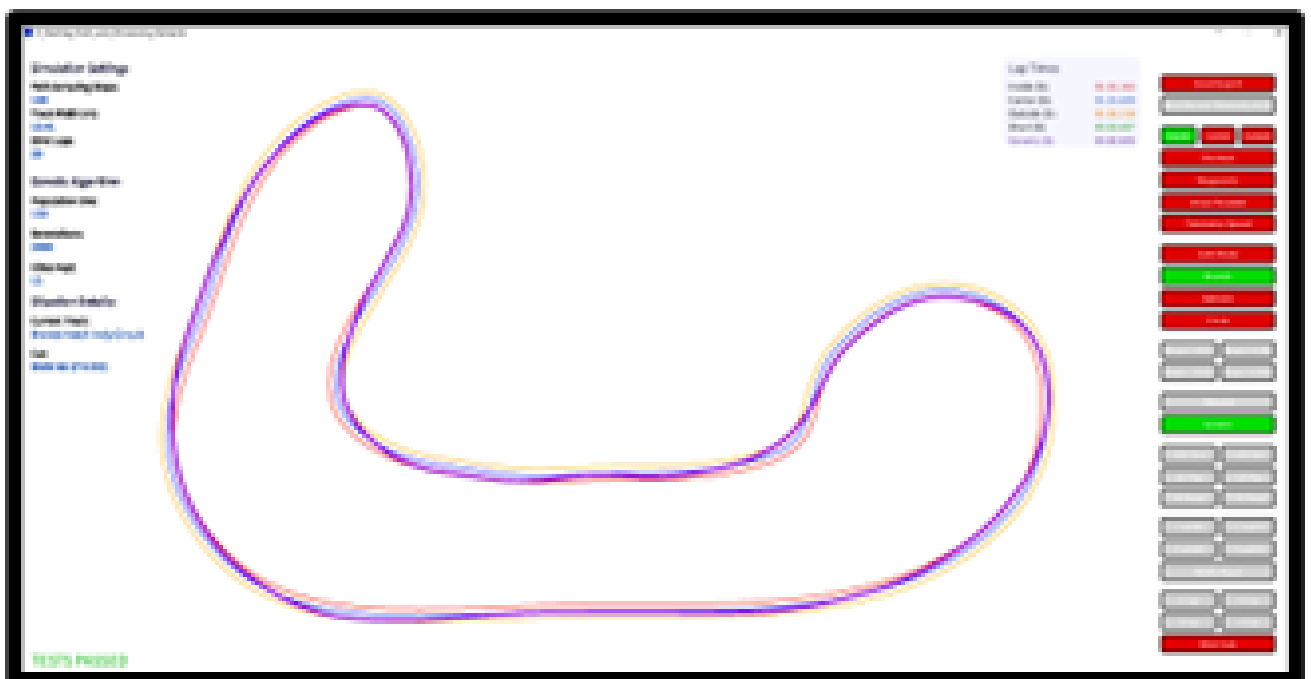


Figure 4.2.2

Appendix 2: Equations Used in the Project

2.1,2.2,2.3 Kinematic Equations

The Kinematic Equations calculate the relationship between initial velocity and final velocity, acceleration, time taken and distance covered, see equations 2.1,2.2, and 2.3.

$$v = u + at \quad (2.1)$$

$$s = u * t + \frac{1}{2}at^2 \quad (2.2)$$

$$v^2 = u^2 + 2as \quad (2.3)$$

where v is the final velocity (ms^{-1}), u is the initial velocity (ms^{-1}), a is the acceleration (ms^{-2}), t is the time taken (s), and s is the displacement of the object (m). (O'Regan, 2000)

2.4 Acceleration Definition

The acceleration equation calculates the acceleration of an object by getting the change in velocity over a set amount of time. See equation 2.4.

$$a = \frac{v-u}{t} \quad (2.4)$$

where v is the final velocity (ms^{-1}), u is the initial velocity (ms^{-1}), a is the acceleration (ms^{-2}), and t is the time taken (s). (O'Regan, 2000)

2.5 Area under the velocity–time graph

Velocity gives the change in distance over time, and displacement is the change in position of an object relative to its initial position. See equation 2.5.

$$s = \int_a^b v dt \quad (2.5)$$

Where a and b define the interval on which we are calculating, v is the velocity of the object in motion (ms^{-1}), and s is the displacement of the object (m). (O'Regan, 2000)

2.6 Determinant

The determinant is a fixed value that measures the signed area of a triangle and allows the calculation of the circumcentre of a circle. See equation 2.6.

$$d = 2(x_1(y_2 - y_3) + x_2(y_3 - y_1) + x_3(y_1 - y_2)) \quad (2.6)$$

Where (x_1, y_1) , (x_2, y_2) and (x_3, y_3) are three points on a curve and d is the resulting determinant. (Weisstein, 2026)

2.7,2.8 Circle Centre

The circumcentre (c_x, c_y) of a circle is a point that is an equal distance from three points (x_1, y_1) , (x_2, y_2) and (x_3, y_3) , meaning that a circle drawn with its centre at the (c_x, c_y) can intersect all three points. See equations 2.7 and 2.8.

$$c_x = \frac{(x_1^2 + y_1^2)(y_2 - y_3) + (x_2^2 + y_2^2)(y_3 - y_1) + (x_3^2 + y_3^2)(y_1 - y_2)}{d} \quad (2.7)$$

$$c_y = \frac{(x_1^2 + y_1^2)(x_3 - x_2) + (x_2^2 + y_2^2)(x_1 - x_3) + (x_3^2 + y_3^2)(x_2 - x_1)}{d} \quad (2.8)$$

Where c_x is the x value of the circumcentre and c_y is the y value of the circumcentre, and (x_1, y_1) , (x_2, y_2) and (x_3, y_3) are the three points on a curve that derive the determinant d in equation 2.5.(Weisstein, 2026)

2.9 Distance between two points

The distance between two points formula allows us to find the displacement between two points. This will allow us to find the radius distance by taking any of the three points of the curve and the circumcentre and getting the displacement. See equation 2.9.

$$s = \sqrt{(x_i - x_{i-1})^2 + (y_i - y_{i-1})^2} \quad (2.9)$$

Where (x_{i-1}, y_{i-1}) and (x_i, y_i) are two points and s is the displacement between them.(whitman.edu, 2026)

2.10 Max Speed around a curve

The maximum speed achievable in a curve is defined by the maximum velocity formula. The formula calculates the maximum speed at a point given the coefficient of friction of the tyre and the track, gravity and the corner radius. See the equation derivation of 2.10.

$$\begin{aligned} 1. \quad F_c &= \frac{mv^2}{r} \quad F_f = \mu N \quad N = mg \\ 2. \quad F_c &= \frac{mv^2}{r} \quad F_f = \mu mg \\ 3. \quad \frac{mv^2}{r} &= \mu mg \\ 4. \quad \frac{v^2}{r} &= \mu g \\ 5. \quad v^2 &= \mu gr \\ 6. \quad v &= \sqrt{\mu \cdot g \cdot r} \end{aligned} \quad (2.10)$$

Where μ represents the coefficient of friction, g is gravity (9.81ms^{-2}), and r is the radius of the curve (m), and $V_{\max}$ is the max speed at that point (ms^{-1}). This equation is obtained by equating the centripetal force with the friction force as derived above, where m is the mass of the vehicle, N is the Newtons of force the vehicle has. F_c is the centripetal force and F_f is the force of friction.(LLC, 2026d; O'Regan, 2000; spark.iop.org, 2026)

2.11 Max Speed

The maximum speed is calculated in equation (2.10), and being stationary, 0ms^{-1} , defines the boundaries of speed at the points between the gates. The simulation will try to accelerate the car to a maximum speed $\leq v_{\max}$, if the car is faster than $v_{\max}$, the simulation will find the latest possible point to brake to be at an appropriate speed using the acceleration formula 2.4. See equation 2.11.

$$v_{car} \leq v_{max} \quad (2.11)$$

Where v_{car} is the car's velocity (ms^{-1}) and v_{max} is the maximum speed, the car can go at that point in the gate without breaking traction.

2.12 Arc Length Accumulation

To get the length of any given lap, the simulation accumulates all the distances between the points in the gates using the distance formula (2.8) and recursion. See equation 2.12.

$$s[i] = s[i - 1] + \sqrt{(x_i - x_{i-1})^2 + (y_i - y_{i-1})^2} \quad (2.12)$$

Where $s[i]$ holds the current total displacement, $s[i-1]$ is the total displacement at the previous gate, and the distance formula (2.8) calculates the displacement from the previous gate to this gate.

2.13 Distance, Speed and Velocity Formula

The Distance, Speed and Velocity formula defines the relationship between speed, distance, and Time. Whereas Speed over a given time will result in each displacement. For this simulation, the formula can be given in three forms. See Equation 2.13.

$$Distance = \frac{Speed}{Time} \Rightarrow Time = \frac{Speed}{Distance} \Rightarrow Speed = Time * Distance \quad (2.13)$$

Where distance is the displacement of the object (m), speed is the rate at which displacement is taking place (ms^{-1}), and time is the time taken for the object to be displaced (s). (O'Regan, 2000)

2.14,2.15 Parametric Spline

The parametric spline is used to smoothly generate a path along (x, y) positions based on arc length. This ensures smooth movement and the avoidance of jagged linear segments. See equations 2.14 and 2.15

$$t_x[i] = Spline(L_x, S, t_s) \quad (2.14)$$

$$t_y[i] = Spline(L_y, S, t_s) \quad (2.15)$$

Where $t_x[i]$ and $t_y[i]$ are the point co-ordinates at point i along the curve within each gate. L_x L_y are the original anchor points that define the gates, and S is the distance along the curve that i represents (m). These can be further read about in sub-chapters 2.5.3 and 2.5.4.

Appendix 3: Use of AI

A2.1 Summary of AI usage

Throughout this project several AI tools were used. In the next sub-chapters, their usage will be discussed.

A2.1.1 Grammarly

Throughout the writing process for this assignment, the most important tool has been Grammarly. The free version of the software was used, and it aided in the use of proper grammar and spelling. It was also extremely useful on picking up on accidental spacing errors and conveying a tone. For a person who is not seasoned at writing academically, Grammarly should be a recommendation for them.

A2.1.2 Chat GPT

Chat GPT 5.4 was utilised during the creation of the genetic algorithm which is cited in A2.2. It was given a detailed prompt and provided a good response which was used as a template to be further developed on. These methods have since been developed however, the template can be still recognised beneath.

As discussed in the Interim report, early in the project Chat GPT 5.2 was utilised to make the excel output methods for the project, however later in the project this was completely removed to make way for a self-made version using Apache.*.

A2.1.3 PowerPoint Designer

Within Microsoft's PowerPoint, there is a feature in which the program will take the text and photographs from the slide and will suggest design ideas. This feature was utilized on several slides to help influence the design.

A2.2 Prompts

A2.2.1 Genetic Algorithm

The bellow Prompt was given to ChatGPT 5.2

“Use this as a basis to create a genetic algorithm.

```
double getGeneticBestLap(double [][][] points, double ratio, int steps)
{
    int generations = 5000;
    int population = 50;
    double [][3] bestTimes = new double [generations][5][3]; //Records 5 best times from each
    generation [generation][time][0=x][1=y][2=time]

    for(int i = 0; i < generations; i++)
    {
        //points[i](where 0 is outer track,1 is center track,2 is inner track)[step along track points n][0
        = x, 1 = y]
        //Ratio is difference from m to pixels

        for(int p = 0; p < population; p++)
        {

            //choose a point along line between x,y at i = 0 and i = 2 for all steps
            //Calculate a Lap Time
            double [ ] radius = new double [steps];
            double [ ] speed = new double [steps];
            for(int j = 0; j < steps; j++)
            {
                //Calculate the radius of the corner from n-1, n, n + 1
                radius [j] = getRadius(x1,y1,x2,y2,x3, y3);

                //Calculate the max speed at each point
                double tyres [ ] = getTyres();
                speed [j] = ((Math.sqrt(((float)tyres[0])*(9.81)*((float)radius [j] [i] [2]))));

                //Get Telemetry by adapting this code for a single lap
                double [ ] telemetry = getTelemetry();

                double laptime = getLaptime();

                if(laptime < bestTimes[i][0][2])
                {
                    //Place it in first and shift it down replicate for all other four positions
                }
            }
        }
    }
}

double [ ] getTelemetry()
{
    double [ ] telemetry;
```

```

/*double[][] carAroundTrack(double[][] speeds, double ratio, int steps)
{
    double accel = getAcceleration();
    double decel = Math.abs(getDeceleration());

    double[][] telemetry = new double[3][steps][4];

    for (int j = 0; j < 3; j++) {
        telemetry[j][0][0] = speeds[j][0][0];
        telemetry[j][0][1] = speeds[j][0][1];
        telemetry[j][0][2] = 0.0;
        telemetry[j][0][3] = 0.0;

        for (int i = 1; i < steps; i++) {
            telemetry[j][i][0] = speeds[j][i][0];
            telemetry[j][i][1] = speeds[j][i][1];

            double dx = telemetry[j][i][0] - telemetry[j][i - 1][0];
            double dy = telemetry[j][i][1] - telemetry[j][i - 1][1];
            double s = Math.sqrt(dx * dx + dy * dy) / ratio;

            double vPrev = telemetry[j][i - 1][2];
            double vCalc = Math.sqrt(vPrev * vPrev + 2.0 * accel * s);
            double vMax = speeds[j][i][3];

            telemetry[j][i][2] = Math.min(vCalc, vMax);
        }

        for (int i = steps - 2; i >= 0; i--) {
            double dx = telemetry[j][i + 1][0] - telemetry[j][i][0];
            double dy = telemetry[j][i + 1][1] - telemetry[j][i][1];
            double s = Math.sqrt(dx * dx + dy * dy) / ratio;

            double vNext = telemetry[j][i + 1][2];
            double vBrake = Math.sqrt(vNext * vNext + 2.0 * decel * s);

            if (vBrake < telemetry[j][i][2]) {
                telemetry[j][i][2] = vBrake;
            }
        }
    }

    return telemetry;
}*/

return telemetry;
}

double getLapTime()
{
    double laptime;

    // double[] lapTime(double[][] telemetry, double[][] speeds, int steps, double ratio)
    //{
    //    double[] times = new double[4][2];

```

```

// for (int j = 0; j < 3; j++) {
//     double totalTime = 0.0;

//     for (int i = 1; i < steps; i++) {
//         double dx = speeds[j][i][0] - speeds[j][i-1][0];
//         double dy = speeds[j][i][1] - speeds[j][i-1][1];

//         double s = Math.sqrt(dx*dx + dy*dy) / ratio;

//         double v = telemetry[j][i][2];

//         if (v > 1e-9) totalTime += s / v;
//     }

//     int minutes = (int)(totalTime / 60.0);
//     double seconds = totalTime - minutes * 60.0;

//     times[j][0] = minutes;
//     times[j][1] = seconds;
// }

// double bestTime = getOptimalLap(telemetry, speeds, steps, ratio);

// int bestMin = (int)(bestTime / 60.0);
// double bestSec = bestTime - bestMin * 60.0;

// times[3][0] = bestMin;
// times[3][1] = bestSec;

// return times;
//}

//double getOptimalLap(double[][][] telemetry, double[][][] speeds, int steps, double ratio)
//{
// final double INF = Double.POSITIVE_INFINITY;

// double[][] dp = new double[3][steps];

// int[][] parent = new int[3][steps];

// for (int j = 0; j < 3; j++) {
//     dp[j][0] = 0.0;
//     parent[j][0] = j;
// }

// for (int i = 1; i < steps; i++) {
//     for (int j = 0; j < 3; j++) {

//         double bestPrev = dp[j][i - 1];
//         int bestlane = j;

//         if (j > 0 && dp[j - 1][i - 1] < bestPrev) {
//             bestPrev = dp[j - 1][i - 1];
//             bestlane = j - 1;

```

```

//    }
//    if (j < 2 && dp[j + 1][i - 1] < bestPrev) {
//        bestPrev = dp[j + 1][i - 1];
//        bestlane = j + 1;
//    }

//    double dx = speeds[j][i][0] - speeds[j][i - 1][0];
//    double dy = speeds[j][i][1] - speeds[j][i - 1][1];
//    double s = Math.sqrt(dx * dx + dy * dy) / ratio;

//    double v = telemetry[j][i][2];
//    double dt = (v > 1e-9) ? (s / v) : INF;

//    parent[j][i] = bestlane;
//    dp[j][i] = bestPrev + dt;
// }
// }
// println();

// int endlane = 0;
// if (dp[1][steps - 1] < dp[endlane][steps - 1]) endlane = 1;
// if (dp[2][steps - 1] < dp[endlane][steps - 1]) endlane = 2;

// double bestTime = dp[endlane][steps - 1];

// int[] optimalj = new int[steps];
// optimalj[steps - 1] = endlane;
// for (int i = steps - 1; i > 0; i--) {
//     optimalj[i - 1] = parent[optimalj[i]][i];
// }
// stroke(0, 255, 0);
// strokeWeight(10);

// for (int i = 1; i < steps; i++) {
//     int jPrev = optimalj[i - 1];
//     int jCurr = optimalj[i];

//     float xPrev = (float) speeds[jPrev][i - 1][0];
//     float yPrev = (float) speeds[jPrev][i - 1][1];
//     float xCurr = (float) speeds[jCurr][i][0];
//     float yCurr = (float) speeds[jCurr][i][1];

//     if(genetic == false)
//     {
//         line(xPrev, yPrev, xCurr, yCurr);
//     }
// }
// stroke(0);
// strokeWeight(2);

// return bestTime;
//}

return laptime

```

}”

A3.1 Code Snippets

A2.1.1 Genetic Algorithm

```
int generations = 2000;
int population = 150;
int elites = 15;

float[][] genes, nextGenes;
double[] currentFitness;
float[] lineX, lineY, v;
float[] geneticVMax;
float[] bestVisualX, bestVisualY;
boolean hasBestLine = false;

double getGeneticBestLap(double[][][] points, double ratio, int steps, double tyre) {
    genes = new float[population][steps];
    nextGenes = new float[population][steps];
    currentFitness = new double[population];
    lineX = new float[steps];
    lineY = new float[steps];
    v = new float[steps];
    geneticVMax = new float[steps];
    bestVisualX = new float[steps];
    bestVisualY = new float[steps];

    for (int p = 0; p < population; p++) {
        for (int j = 0; j < steps; j++) {
            genes[p][j] = random(0.0f, 1.0f);
        }
        smoothGenesInPlace(genes[p], steps, 5);
    }

    double globalBest = Double.MAX_VALUE;

    for (int i = 0; i < generations; i++) {
        for (int p = 0; p < population; p++) {
            generateSplinePath(genes[p], points, steps);
            calculatePhysics((float)tyre, steps, ratio);
            getTelemetry((float)getAcceleration(), (float)getDeceleration(), (float)ratio, steps);
            currentFitness[p] = getLapTime((float)ratio, steps);

            if (currentFitness[p] < globalBest) {
                globalBest = currentFitness[p];
                updateBestVisual(steps);
            }
        }
        evolve(steps, i);
    }
    return globalBest;
}

void generateSplinePath(float[] genome, double[][][] points, int steps) {
    for (int i = 0; i < steps; i++) {
        lineX[i] = lerp((float)points[0][i][0], (float)points[2][i][0], genome[i]);
    }
}
```

```

    lineY[i] = lerp((float)points[0][i][1], (float)points[2][i][1], genome[i]);
}
}

void calculatePhysics(float mu, int steps, double ratio) {
    for (int j = 0; j < steps; j++) {
        int prev = (j - 1 + steps) % steps;
        int next = (j + 1) % steps;
        double r = getRadius(lineX[prev], lineY[prev], lineX[j], lineY[j], lineX[next], lineY[next], ratio);
        geneticVMax[j] = sqrt(mu * 9.81f * (float)r);
    }
    geneticVMax[steps - 1] = 400;
}

void getTelemetry(float acc, float dec, float ratio, int steps) {
    v[0] = 0.0f;
    for (int i = 1; i < steps; i++) {
        float s = dist(lineX[i-1], lineY[i-1], lineX[i], lineY[i]) / ratio;
        v[i] = min(sqrt(sq(v[i-1]) + 2 * acc * s), geneticVMax[i]);
    }
    for (int i = steps - 2; i >= 0; i--) {
        float s = dist(lineX[i], lineY[i], lineX[i+1], lineY[i+1]) / ratio;
        float vBrake = sqrt(sq(v[i+1]) + 2 * dec * s);
        if (vBrake < v[i]) v[i] = vBrake;
    }
}

double getLapTime(float ratio, int steps) {
    double total = 0;
    for (int i = 1; i < steps; i++) {
        float s = dist(lineX[i-1], lineY[i-1], lineX[i], lineY[i]) / ratio;
        total += (v[i] > 0.1) ? (s / v[i]) : 1000;
    }
    return total;
}

void updateBestVisual(int steps) {
    if (bestVisualX == null || bestVisualX.length != steps) {
        bestVisualX = new float[steps];
        bestVisualY = new float[steps];
    }
    arrayCopy(lineX, bestVisualX);
    arrayCopy(lineY, bestVisualY);
    hasBestLine = true;
}

void drawBestLine() {
    if (!hasBestLine) return;
    noFill();
    stroke(147, 0, 211);
    strokeWeight(4);
    beginShape();
    for (int i = 0; i < bestVisualX.length; i++) {
        curveVertex(bestVisualX[i], bestVisualY[i]);
    }
}

```

```

    endShape(CLOSE);
    strokeWeight(2);
}

double getRadius(float x1, float y1, float x2, float y2, float x3, float y3, double ratio) {
    double a = dist(x1, y1, x2, y2);
    double b = dist(x2, y2, x3, y3);
    double c = dist(x3, y3, x1, y1);
    double area = abs(x1*(y2-y3) + x2*(y3-y1) + x3*(y1-y2)) * 0.5;
    double hold = (area < 0.01) ? 10000 : (a * b * c) / (4 * area);
    hold = hold / ratio;
    return hold;
}

void evolve(int steps, int currentGen) {
    float progress = (float)currentGen / generations;
    float power = lerp(0.15f, 0.02f, progress);
    int[] order = argsortByFitness(currentFitness);

    for (int e = 0; e < elites; e++) {
        int srcIndex = order[e];
        arrayCopy(genes[srcIndex], nextGenes[e]);
    }

    for (int idx = elites; idx < population; idx++) {
        int p1 = tournamentPick(currentFitness, 6);
        int p2 = tournamentPick(currentFitness, 6);
        crossoverArithmetic(genes[p1], genes[p2], nextGenes[idx], steps);
        mutate(nextGenes[idx], steps, 0.08f, power);
        if (random(1) < 0.4f) {
            smoothGenesInPlace(nextGenes[idx], steps, 3);
        }
    }

    for (int p = 0; p < population; p++) {
        arrayCopy(nextGenes[p], genes[p]);
    }
}

int tournamentPick(double[] fitness, int k) {
    int best = (int)random(population);
    for (int i = 1; i < k; i++) {
        int contender = (int)random(population);
        if (fitness[contender] < fitness[best]) {
            best = contender;
        }
    }
    return best;
}

void crossoverArithmetic(float[] p1, float[] p2, float[] child, int steps) {
    float weight = random(0.2, 0.8);
    for (int i = 0; i < steps; i++) {
        child[i] = (p1[i] * weight) + (p2[i] * (1.0f - weight));
    }
}

```

```

}

void mutate(float[] g, int steps, float rate, float power) {
    for (int i = 0; i < steps; i++) {
        if (random(1) < rate) {
            float change = random(-power, power);
            int window = 5;
            for(int w = -window; w <= window; w++) {
                int idx = (i + w + steps) % steps;
                g[idx] += change * (1.0 - abs(w)/(float)window);
                g[idx] = constrain(g[idx], 0.0f, 1.0f);
            }
        }
    }
}

int[] argsortByFitness(double[] fitness) {
    int[] indices = new int[population];
    for (int i = 0; i < population; i++) indices[i] = i;
    for (int i = 0; i < population - 1; i++) {
        for (int j = i + 1; j < population; j++) {
            if (fitness[indices[j]] < fitness[indices[i]]) {
                int temp = indices[i];
                indices[i] = indices[j];
                indices[j] = temp;
            }
        }
    }
    return indices;
}

void smoothGenesInPlace(float[] g, int steps, int passes) {
    for (int p = 0; p < passes; p++) {
        for (int i = 0; i < steps; i++) {
            g[i] = (g[(i-1+steps)%steps] + g[i] + g[(i+1)%steps]) / 3.0f;
        }
    }
}

```